00. Table of Contents

This document provides a comprehensive overview of the course sections and subsections, mapping the theoretical foundations to low-level implementations in C++, FastFlow, MPI, and CUDA.

1. Foundations of Parallel Computing & Performance Measures **Metrics:** Latency (L), Service Time (T_s), Completion Time (T_c), Speedup (S_p), Efficiency (E). **Laws:** Amdahl's Law, Gustafson's Law. **Patterns:** Pipeline, Farm (Master-Worker).
2. Structured Parallel Programming & Algorithmic Skeletons **Framework:** Separation of concerns, Cole's Algorithmic Skeletons. **Stream Parallel:** `pipe`, `farm`. **Data Parallel:** `map`, `reduce`.
3. Shared Memory & Low-Level Concurrency **Architecture:** Cache Coherence (MESI), Synchronization, Branch Divergence (Predication). **C++ Implementation:** `std::thread`, data races, OpenMP directives.
4. Thread Affinity & Memory Consistency **Hardware Interaction:** Thread migration penalties, NUMA, enforcing affinity (`sched_setaffinity`). **Memory Models:** Sequential Consistency, Relaxed models, C++11 `std::atomic` and memory ordering. **Optimization:** Mitigating false sharing via memory alignment.
5. Distributed Systems & Message Passing **Paradigm:** Isolated memory spaces, synchronous vs asynchronous communication. **MPI:** `MPI_Comm_world`, Ranks, Point-to-Point (`MPI_Send`, `MPI_Irecv`), Collective operations (`MPI_Bcast`, `MPI_Reduce`).
6. FastFlow Framework & Stream Processing **Design:** Lock-free/wait-free queues (SPSC), zero-copy routing. **Components:** `ff_node`, `svc` method, EOS/GO_ON tokens. **C++ Skeletons:** `ff_Pipe`, `ff_Farm`.
7. Data Parallelism & GPU Concurrency **FastFlow:** `ff::ParallelFor`, `ff::ParallelForReduce`. **GPU Basics:** SIMT execution, Threads/Blocks/Grids hierarchy. **CUDA:** `__global__` kernels, Global/Shared/Register memory, `cudaMemcpy`.
8. Advanced GPU Concurrency & CUDA Optimizations **Execution:** Warp divergence, Global Memory Coalescing (AoS vs SoA). **Optimization:** Tiling with `__shared__` memory, `__syncthreads()`. **Latency Hiding:** Asynchronous execution with CUDA Streams.
9. Communication Cost Models & Macro-Dataflow **Analytical Models:** $T_{comm}(L) = t_{setup} + \frac{L}{B_w}$, impact on scalability. **MDF Architecture:** Instruction Pool, Matching Store, Ready Queue, Critical Path weighting.
10. Distributed Skeletons Implementation **MPI Pipeline:** Static mapping, data batching. **MPI Farm:** Master-Worker architecture, static vs on-demand dynamic load balancing.
11. Advanced Patterns & Stateful Skeletons **Patterns:** Divide & Conquer (Thread pools, cut-off depth), Stencil (Ghost cells, Halo exchange). **State Management:** Key-based routing for stateful farm workers.
12. Formal Semantics of Parallel Constructs **Methods:** Labeled Transition Systems (LTS), Operational Semantics for streams. **Equivalence:** Normal Form rewriting rules, Bisimulation for non-deterministic behavior.
13. Autonomic Computing & Quality of Service (QoS) **Adaptability:** Under/Over-provisioning. **Control Loop:** MAPE (Monitor, Analyze, Plan, Execute). **Implementation:** Dynamic thread pools, hysteresis, `ff_aFarm`.

01. Foundations of Parallel Computing & Performance Measures

This chapter introduces the fundamental concepts, performance metrics, and basic patterns (skeletons) used in parallel computing to evaluate and structure concurrent execution.

1. Performance Metrics

To evaluate the quality and efficiency of a parallel system, we rely on several quantitative metrics:

1.1 Latency (L)

Latency is the time required to complete a single task from its beginning to its end. In a system composed of multiple stages (e.g., a pipeline), it is the sum of the times spent in each individual stage.

1.2 Service Time (T_s)

Service Time is the time interval between the emission of two consecutive results. It dictates the maximum throughput of the system. * In a sequential execution, T_s is equal to the Latency. * In a parallel system (like a pipeline), the overall service time is bottlenecked by the slowest stage: $T_s = \max\{T_{s_1}, T_{s_2}, \ldots, T_{s_k}\}$.

1.3 Completion Time (T_c)

The total time required to process m tasks. Given a pipeline or a generic parallel system, the initial phase involves filling the system (paying the latency L), and subsequently, results are emitted every T_s.

$$T_c = L + (m-1)T_s$$

For a very large number of tasks ($m \gg 1$), the Completion Time can be approximated as $T_c \approx mT_s$.

1.4 Speedup (S_p)

Speedup measures the relative performance gain of executing a program on n parallel processing elements compared to a sequential execution.

$$S_p(n) = \frac{T_{seq}}{T_{par}(n)}$$

Where T_{seq} is the completion time of the best sequential algorithm and $T_{par}(n)$ is the completion time of the parallel algorithm using n workers. * Ideally, we seek **linear speedup** ($S_p(n) = n$). * Speedup is theoretically bounded by the critical path: $S_p \leq \frac{\text{Work}}{\text{Span}}$.

1.5 Efficiency (E)

Efficiency measures how well the parallel resources are being utilized. It is defined as the ideal parallel time divided by the actual parallel time, or equivalently, Speedup divided by the number of workers n.

$$E(n) = \frac{T_{id}(n)}{T(n)} = \frac{T_{seq}}{n \cdot T_{par}(n)} = \frac{S_p(n)}{n}$$

An ideal efficiency is 1 (or 100%), corresponding to linear speedup.

---

2. Laws of Scalability

2.1 Amdahl's Law

Amdahl's law provides a theoretical upper bound on the speedup of a program when only a fraction of it can be parallelized. If f is the strictly sequential fraction of the program (e.g., $10\% = 0.10$), the maximum speedup is bounded by $\frac{1}{f}$ (e.g., $10\times$) regardless of how many cores n are added.

$$S_p(n) \leq \frac{1}{f + \frac{1-f}{n}} \xrightarrow{n\to\infty} \frac{1}{f}$$

2.2 Gustafson's Law

Gustafson's Law counteracts the pessimism of Amdahl's Law by observing that as computing power increases, the size of the problem typically increases as well. The work required for the parallel part grows much faster than the sequential part. If the sequential fraction f decreases as the data size grows, the system can achieve near-linear speedup for massive datasets.

---

3. Basic Parallel Patterns

Parallel patterns (or algorithmic skeletons) provide high-level abstractions to structure concurrent computations, avoiding low-level lock and thread management.

3.1 Pipeline

A computation is divided into k sequential stages connected by communication channels. Each stage processes a data item and passes it to the next. * **Service Time (T_s):** Bounded by the slowest stage. $T_s = \max_{i=1\ldots k}\{T_{s_i}\}$. * **Latency ($L$):** Sum of the times of all stages. $L = \sum_{i=1}^{k} T_{s_i}$. * **Optimization:** If a stage is a bottleneck, its service time can be lowered by parallelizing that specific stage internally (e.g., turning it into a Farm).

3.2 Farm (Task Farm / Master-Worker)

A Farm replicates the execution of a single function over multiple workers. It is ideal for embarrassingly parallel workloads where tasks are completely independent. * **Structure:** Consists of an *Emitter* (Scheduler) that distributes tasks, a set of *Workers* that compute the tasks independently, and a *Collector* that gathers the results. * **Service Time (T_s):** If the sequential service time is T_w and we have n_w workers, the ideal parallel service time is $T_s = \frac{T_w}{n_w}$. * **Efficiency:** A farm is highly efficient, but scheduling overhead and the sequential nature of Emitter/Collector can become bottlenecks if the communication time dominates the computation time.

---

4. Hardware & Context

When designing parallel applications, the underlying hardware heavily influences the chosen mechanism and abstraction: * **Memory Architectures:** Threads can communicate via Shared Memory (requiring synchronization primitives like locks, condition variables, or barriers) or via Message Passing (Point-to-Point or Collective communications). * **Grain Size (Granularity):** The ratio of computation time to communication time. A coarse-grained computation (high computation-to-communication ratio) is generally necessary to offset the overhead of data splitting and dispatching. * **Excess Parallelism (Oversubscription):** Spawning more parallel activities than the available hardware contexts. It allows the OS/runtime to hide latency (e.g., if one thread blocks for I/O, another can be scheduled), avoiding idle cores.

02. Structured Parallel Programming & Algorithmic Skeletons

This chapter delves into Structured Parallel Programming (SPP) and Algorithmic Skeletons. The core philosophy of this approach is the **separation of concerns**: isolating the purely sequential computational logic (business logic) from the parallel coordination and communication mechanisms.

1. The Algorithmic Skeleton Framework

Introduced by Murray Cole, **Algorithmic Skeletons** are recurring parallel paradigms abstracted into reusable, parameterizable templates. They provide high-level parallel constructs that hide the underlying complexity of thread management, synchronization, and message passing.

Skeletons are typically divided into two macro-categories based on the nature of the parallelism they exploit:

1. **Stream Parallel Skeletons:** Operate on streams of data items (tasks) flowing through the system. Examples: `pipeline`, `farm`.
2. **Data Parallel Skeletons:** Operate on large, partitioned data structures where the same computation is applied to different chunks simultaneously. Examples: `map`, `reduce`, `stencil`.

---

2. Stream Parallel Skeletons

Stream parallel skeletons model systems that process a continuous flow of input tasks. The execution is driven by the availability of new data tokens.

2.1 The Pipeline Skeleton (`pipe`)

The `pipe` skeleton connects k stages $S_1, S_2, \ldots, S_k$ in a sequence. The output stream of stage S_i becomes the input stream of stage S_{i+1}.

- **Semantics:** $\text{pipe}(S_1, S_2)(x) = S_2(S_1(x))$ extended over a stream.
- **Performance Model:**
  - **Service Time (T_s):** Bottlenecked by the slowest stage. $T_s = \max_{i=1\ldots k}\{T_{S_i}\}$
  - **Latency (L):** The sum of the times taken by all individual stages. $L = \sum_{i=1}^{k} T_{S_i}$
- **Optimization:** If a stage is identified as a bottleneck, it must be parallelized (e.g., by wrapping it in a `farm` or a nested `pipe`) to lower its service time and improve the overall throughput.

2.2 The Farm Skeleton (`farm`)

The `farm` skeleton models independent task execution (embarrassingly parallel). It consists of three entities:

1. **Emitter (E):** Fetches data from the input stream and dispatches it to the workers.
2. **Workers ($W_1 \ldots W_n$):** A pool of threads/processes computing the identical function f on different input data.
3. **Collector (C):** Gathers results from the workers and forwards them to the output stream.

- **Performance Model:**
  - Let T_E be the time the Emitter takes to schedule a task, T_W the time a worker takes to compute f, and T_C the time the Collector takes to gather a result.
  - **Service Time (T_s):** $T_s = \max\left\{T_E, \frac{T_W}{n}, T_C\right\}$
  - **Latency (L):** $L = T_E + T_W + T_C$
- **Scalability Bottleneck:** The Emitter and Collector are strictly sequential components. As the number of workers n increases, $\frac{T_W}{n}$ decreases. Eventually, T_E or T_C will become the maximum value, causing the speedup to plateau. The maximum useful number of workers is $n_{max} = \frac{T_W}{\max(T_E, T_C)}$.

---

3. Data Parallel Skeletons

Data parallel skeletons apply operations across partitions of a single, large data structure (like vectors, matrices, or graphs).

3.1 The Map Skeleton (`map`)

The `map` skeleton takes an array (or list) and applies a purely sequential function f to each element independently.

- **Semantics:** $\text{map}(f)([x_1, x_2, \ldots, x_n]) = [f(x_1), f(x_2), \ldots, f(x_n)]$
- **Execution:** The runtime partitions the array into chunks, assigns chunks to different processing elements (PEs), computes the chunks in parallel, and merges the results.
- **Completion Time (T_c):** For an array of size N distributed over n workers, the ideal completion time is $T_c = \frac{N \cdot T_f}{n} + T_{scatter} + T_{gather}$.

3.2 The Reduce Skeleton (`reduce`)

The `reduce` skeleton takes an array and aggregates its elements into a single value using a binary operator $\oplus$. For the skeleton to yield deterministic results regardless of the parallel execution tree, the operator $\oplus$ **must be associative**.

- **Semantics:** $\text{reduce}(\oplus)([x_1, x_2, \ldots, x_n]) = x_1 \oplus x_2 \oplus \cdots \oplus x_n$
- **Execution:** Typically implemented using a parallel tree reduction.
- **Completion Time (T_c):** With n workers, the reduction tree has a depth of $\log_2(n)$. Thus, parallel time scales logarithmically rather than linearly in the number of processors.

---

4. Operational Semantics for Skeletons

To formalize the behavior of these skeletons without ambiguity, we use operational semantics modeled as transition systems. States represent the current configuration of the computation, and transitions ($\rightarrow$) describe computation steps.

Let $\langle x, \tau \rangle$ represent an input stream where x is the current token and τ is the rest of the stream. Let Δ represent the inner computation of a skeleton (e.g., the worker function in a farm).

1. **Sequential execution (`seq`):**

   Applies a function f to a stream.

$$\langle \text{seq } f, \langle x, \tau \rangle \rangle \rightarrow \langle f(x) \rangle :: \langle \text{seq } f, \langle \tau \rangle \rangle$$

2. **Farm execution (`farm`):**

   The farm processes the first element of the stream and concurrently processes the rest.

$$\langle \text{farm } \Delta, \langle x, \tau \rangle \rangle \rightarrow \langle \Delta(x) \rangle \parallel \langle \text{farm } \Delta, \langle \tau \rangle \rangle$$

   (Note: The exact formalization can include explicit rules for Emitter scheduling and Collector gathering depending on the specific transition system used).

---

5. Implementation in Real Frameworks (e.g., C++ FastFlow)

In modern C++ environments, implementing these mathematical models effectively requires low-overhead runtime supports. Frameworks like FastFlow implement stream parallel skeletons using **lock-free** and **wait-free** queues (e.g., Single-Producer Single-Consumer queues for pipelines).

- In a FastFlow `ff_pipeline`, each stage is a C++ object executing a sequential loop, pulling data from a lock-free input queue and pushing to an output queue.
- In a `ff_farm`, the Emitter acts as a load balancer, distributing pointers to memory allocations rather than copying the data itself, thereby minimizing T_E and extending the scalability limits defined by Amdahl's Law.

03. Shared Memory & Low-Level Concurrency

This chapter explores the intricacies of Shared Memory architectures, the challenges of keeping memory consistent across multiple processors, the severe impact of memory latency, and the practical implementation of concurrent execution using modern tools.

1. Shared Memory Architectures

In a shared memory system, multiple processing elements (PEs) have access to a single, global memory address space. While this simplifies the programming model (data does not need to be explicitly sent and received via messages), it introduces severe hardware and software complexities.

1.1 The Memory Hierarchy and Latencies

To understand performance bottlenecks in shared memory, one must analyze the memory hierarchy. CPUs use layers of smaller, faster memory to hide the massive latency of the main RAM. The typical access times dictate the absolute necessity of data locality:

- **L1 Cache:** Private to each core. Extremely fast (typically ~1-2 ns) but very small (e.g., 32KB-64KB).
- **L2 Cache:** Usually private per core, sometimes shared between a pair. Slower (typically ~3-10 ns) but larger (e.g., 256KB-1MB).
- **L3 Cache:** Shared across all cores on the processor die. Slower still (~10-40 ns) but significantly larger (e.g., 8MB-64MB).
- **Main Memory (DRAM):** The global shared address space. Massive capacity but severe latency (often ~100 ns or more).

When a thread requests data, the CPU checks these caches sequentially. A cache miss at all levels forces the CPU to stall for hundreds of clock cycles while waiting for data from DRAM.

1.2 Cache Coherence

Because modern CPUs heavily utilize these multiple levels of cache to hide DRAM latency, if multiple cores load the same memory block into their local L1 caches and one core modifies it, the other copies become stale.

- **Coherence Protocols:** The hardware must keep all these copies consistent and coherent through specific protocols (e.g., MESI - Modified, Exclusive, Shared, Invalid).
- **Performance Impact:** When a core writes to a shared variable, the coherence protocol must invalidate or update the cached copies in other cores. If another core needs that variable, it suffers a severe cache miss penalty, forcing a slow fetch. This generates heavy bus traffic and latency, a phenomenon known as **False Sharing** if cores are writing to independent, totally different variables that just happen to reside on the exact same 64-byte cache line.

1.3 Synchronization Primitives

To prevent race conditions when multiple threads access shared data, we must enforce mutual exclusion using synchronization mechanisms:

- **Mutexes and Locks:** Ensure that only one thread can execute a critical section at a time. However, locking forces sequential execution, drastically impacting Amdahl's Law and adding OS-level context switching overhead.

- **Atomics and Lock-Free Programming:** Utilizing hardware-level atomic instructions (like Compare-And-Swap) to update variables without blocking threads. This is crucial for high-performance concurrent data structures.

---

2. Branch Divergence and Predication

When executing parallel workloads on architectures with SIMT (Single Instruction, Multiple Threads) execution models, such as GPUs or highly vectorized CPU units, control flow becomes a critical bottleneck.

2.1 The Issue with Branching

If threads executing in lockstep encounter a conditional branch (e.g., an `if-else` statement) and diverge (some take the `true` path, others the `false` path), the hardware must serialize the execution. It first executes the `true` path for the active threads, masking the others, and then executes the `false` path. This halves the parallel efficiency.

2.2 Predication as a Solution

To mitigate branch divergence, one technique is **predication**. Instead of branching, the hardware computes both paths simultaneously.

- **How it works:** All threads compute the instructions for both the `if` and the `else` branches. The results are guarded by a boolean predicate (flag). Only the results associated with the `true` predicate are actually committed to memory or registers.
- **Trade-off:** This requires doing extra computational work, but it avoids the severe penalty of breaking the instruction pipeline and serializing the parallel execution context.

---

3. Concurrency in Modern C++

Modern standards provide robust abstractions for managing shared memory parallelism without relying on raw OS-level threads (like POSIX threads).

3.1 Standard Threads and Data Races

Managing threads directly requires explicit synchronization.

```
#ifndef SHARED_MEMORY_THREADS_H
#define SHARED_MEMORY_THREADS_H


#include <iostream>
#include <thread>
#include <vector>
#include <mutex>

std::mutex mtx;

void compute_chunk(int thread_id, int& shared_accumulator) {
    // compute local aggregate to minimize lock contention
    int local_sum = 0;
    for (int i = 0; i < 1000; ++i) {
        local_sum += (i * thread_id);
    }
```

```
    // update shared state safely
    std::lock_guard<std::mutex> lock(mtx);
    shared_accumulator += local_sum;

    std::cout << "THREAD " << thread_id << " FINISHED EXECUTION. CURRENT ACCUMULATOR: " << shared_accumu
}

int main() {
    int shared_accumulator = 0;
    std::vector<std::thread> pool;

    // spawn threads
    for (int i = 0; i < 4; ++i) {
        pool.emplace_back(compute_chunk, i, std::ref(shared_accumulator));
    }

    // wait for completion
    for (auto& t : pool) {
        t.join();
    }

    std::cout << "FINAL RESULT COMPUTED SUCCESSFULLY\n";
    return 0;
}

#endif // SHARED_MEMORY_THREADS_H
```

3.2 OpenMP

For data-parallel loops and regular computational kernels, OpenMP provides compiler directives (`#pragma`) that dramatically simplify parallelization by handling thread creation, work distribution, and synchronization automatically.

```
#ifndef SHARED_MEMORY_OPENMP_H
#define SHARED_MEMORY_OPENMP_H

#include <iostream>
#include <vector>
#include <omp.h>

void process_array(std::vector<int>& data) {
    int total_sum = 0;

    // parallelize the loop and perform a reduction on total_sum
    #pragma omp parallel for reduction(+:total_sum)
    for (size_t i = 0; i < data.size(); ++i) {
        data[i] = data[i] * 2;
        total_sum += data[i];
    }

    std::cout << "OPENMP REDUCTION COMPLETED. SUM: " << total_sum << "\n";
}

#endif // SHARED_MEMORY_OPENMP_H
```

By leveraging these constructs, developers can bridge the gap between abstract algorithmic skeletons (like Map and Reduce) and actual high-performance execution on shared memory architectures.

04. Thread Affinity & Memory Consistency

This chapter explores the low-level interactions between the operating system, the hardware architecture, and concurrent programs. Understanding how threads are scheduled and how memory is accessed is crucial for extracting maximum performance from multi-core systems.

1. Thread and Process Affinity

When the operating system's scheduler manages a pool of threads, it is free to migrate them across different physical cores to balance the system load. While this is optimal for general-purpose computing, it introduces severe performance penalties for High-Performance Computing (HPC) and structured parallel applications.

1.1 The Cost of Thread Migration

If a thread is executing on Core C_0 and is migrated to Core C_1, the following issues arise:

- **Cache Invalidation:** The L1 and L2 caches of C_0 contain the hot data for the thread. When moved to C_1, the thread experiences a burst of cache misses, forcing the CPU to fetch data from the slower L3 cache or Main Memory.
- **Context Switch Overhead:** The registers, program counter, and translation lookaside buffer (TLB) state must be saved and restored.
- **NUMA Penalties:** In Non-Uniform Memory Access (NUMA) architectures, memory is divided into nodes assigned to specific sockets. If a thread is moved to a different socket, accessing its originally allocated memory becomes significantly slower due to inter-socket communication links (e.g., Intel UPI, AMD Infinity Fabric).

1.2 Enforcing Affinity

To prevent migration, we can explicitly bind threads to specific hardware contexts (cores or hyper-threads). This is known as **Thread Affinity**.

In Linux environments, this is often handled using the `sched_setaffinity` system call or through library wrappers. In modern C++, we access the underlying OS handle via `.native_handle()`.

2. Memory Consistency Models

Cache coherence (discussed in the previous chapter) ensures that all cores see the same value for a *single* memory location. However, it does not guarantee the order in which memory operations across *multiple* locations become visible to different cores. This is dictated by the **Memory Consistency Model**.

2.1 Sequential Consistency (SC)

The most intuitive model, proposed by Lamport. A system is sequentially consistent if the result of any execution is the same as if the operations of all processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.

- **Drawback:** Enforcing SC completely prevents the compiler and the CPU from reordering memory instructions for optimization (e.g., storing to a write buffer while a load completes).

2.2 Relaxed Memory Models

To achieve high performance, modern hardware (like x86, and particularly ARM) reorders loads and stores.

To write correct lock-free code, developers must use **Memory Barriers** (or fences) to explicitly enforce ordering where necessary.

2.3 C++11 Memory Model

C++ provides a standard way to deal with memory ordering using `std::atomic` and `std::memory_order` flags:

- `memory_order_relaxed`: No synchronization or ordering constraints, only atomicity for the specific operation.
- `memory_order_acquire` / `memory_order_release`: A store with `release` synchronizes with a load with `acquire` on the same atomic variable. All memory writes before the release become visible to the thread doing the acquire.
- `memory_order_seq_cst`: Enforces strict sequential consistency (the default, but most expensive).

3. Mitigating False Sharing

As introduced previously, false sharing occurs when independent variables accessed by different threads reside on the same cache line (typically 64 bytes).

To solve this, we must enforce spatial separation of data in memory using alignment directives.

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <atomic>
#include <pthread.h>

// align to 64 bytes to prevent false sharing between array elements
struct alignas(64) thread_state {
    std::atomic<int> counter;
};

void compute_work(int id, thread_state* state) {
    // get native handle to pin thread to specific cpu core
    pthread_t thread = pthread_self();
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
    CPU_SET(id, &cpuset);

    // enforce affinity
    pthread_setaffinity_np(thread, sizeof(cpu_set_t), &cpuset);

    // execute workload using relaxed memory order for pure counters
    for(int i = 0; i < 100000; ++i) {
        state[id].counter.fetch_add(1, std::memory_order_relaxed);
    }
}

int main() {
    const int num_threads = 4;
    // allocated contiguous memory, but alignas guarantees 64-byte spacing
```

```
    thread_state states[num_threads];
    std::vector<std::thread> pool;

    for (int i = 0; i < num_threads; ++i) {
        states[i].counter.store(0, std::memory_order_relaxed);
        pool.emplace_back(compute_work, i, states);
    }

    for (auto& t : pool) {
        t.join();
    }

    return 0;
}
```

By manually orchestrating memory alignment and thread placement, the abstractions defined by parallel skeletons can be implemented with minimal hardware-level interference, achieving near-linear speedups bounded only by Amdahl's fraction.

05. Distributed Systems & Message Passing

This chapter shifts the focus from shared memory systems to distributed memory architectures, exploring how isolated processes communicate and synchronize over a network using the Message Passing paradigm, with a deep dive into the MPI standard.

1. Distributed Memory Architectures

Unlike shared memory systems where all cores access a single global address space, distributed systems consist of independent processing nodes. Each node has its own local memory, CPU, and operating system instance.

- **No Cache Coherence Issues:** Since memory is isolated, there is no hardware-level cache coherence protocol spanning across nodes. False sharing is strictly a local, intra-node issue.
- **Communication via Network:** Nodes communicate explicitly by sending and receiving messages over an interconnection network (e.g., Ethernet, InfiniBand).
- **Latency and Bandwidth:** Network communication is orders of magnitude slower than memory access. Granularity must be coarse; overlapping computation and communication becomes critical to hide network latency.

2. The Message Passing Paradigm

In the message passing model, concurrent activities (processes) interact by explicitly exchanging data payloads.

2.1 Send and Receive Primitives

The foundational operations are `send(dest, message)` and `receive(src, message)`. These primitives can be classified based on their synchronization semantics:

- **Synchronous (Blocking):** The sender blocks until the receiver has acknowledged the receipt of the message. This naturally enforces a synchronization barrier between the two processes.
- **Asynchronous (Non-Blocking):** The sender dispatches the message to a buffer (managed by the OS or network hardware) and immediately resumes execution. The receiver can fetch the message from the buffer later. This allows computation-communication overlap but requires complex buffer management.

3. Message Passing Interface (MPI)

MPI is the de-facto standard for High-Performance Computing (HPC) on distributed systems. It defines a rich API for process communication, defining a static set of processes spawned at the beginning of the execution.

3.1 Communicators and Ranks

- **Communicator (`MPI_Comm`):** A communication universe. `MPI_COMM_WORLD` is the default communicator encompassing all spawned processes.
- **Rank:** A unique integer identifier assigned to each process within a communicator, ranging from 0 to $N - 1$. Ranks are used to target destinations and identify sources.

3.2 Point-to-Point Communication

Direct communication between two specific ranks. * `MPI_Send`: Basic blocking send. Returns when the application buffer can be safely reused. * `MPI_Recv`: Basic blocking receive. Waits until a matching message arrives. * `MPI_Isend` / `MPI_Irecv`: Non-blocking variants. They return an `MPI_Request` handle that must be explicitly checked or waited upon later (using `MPI_Wait`).

3.3 Collective Communication

Operations that involve all ranks within a communicator simultaneously. They are heavily optimized for the specific network topology (e.g., using hypercube routing or tree-based reductions). * **Broadcast (`MPI_Bcast`):** Rank 0 sends the same data to all other ranks. * **Scatter (`MPI_Scatter`):** Rank 0 splits an array into chunks and sends one chunk to each rank. * **Gather (`MPI_Gather`):** All ranks send their local data chunks to Rank 0, which concatenates them. * **Reduce (`MPI_Reduce`):** All ranks provide a value, and an associative operator (like `MPI_SUM`, `MPI_MAX`) aggregates them into a single result at the root rank.

---

4. MPI Implementation in C++

Below is an implementation of a standard scatter-compute-gather pattern using MPI primitives.

```cpp
#include <iostream>
#include <vector>
#include <mpi.h>

int main(int argc, char** argv) {
    // initialize mpi environment
    MPI_Init(&argc, &argv);

    int rank;
    int size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    const int elements_per_proc = 1000;
    std::vector<int> local_data(elements_per_proc);
    std::vector<int> global_data;

    if (rank == 0) {
        std::cout << "MASTER NODE INITIALIZED WITH " << size << " PROCESSES" << std::endl;
        global_data.resize(size * elements_per_proc);
```

```
        // initialize dummy workload
        for (int i = 0; i < size * elements_per_proc; ++i) {
            global_data[i] = 1;
        }
    }

    // scatter data from root to all processes
    MPI_Scatter(global_data.data(), elements_per_proc, MPI_INT,
                local_data.data(), elements_per_proc, MPI_INT,
                0, MPI_COMM_WORLD);

    // compute local workload
    int local_sum = 0;
    for (int i = 0; i < elements_per_proc; ++i) {
        local_sum += local_data[i];
    }

    // reduce local results to global sum on root
    int global_sum = 0;
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        std::cout << "COMPUTATION FINISHED. GLOBAL SUM: " << global_sum << std::endl;
    }

    // tear down mpi environment
    MPI_Finalize();
    return 0;
}
```

4.1 Overlapping Computation and Communication

To squeeze maximum performance, advanced MPI code utilizes `MPI_Isend` and `MPI_Irecv`. While the network interface controller (NIC) is transferring the boundary data asynchronously, the CPU computes the inner core of the local dataset. Once the inner core is done, the code waits for the boundary transfers to complete (`MPI_Wait`) before computing the edges. This effectively hides the network latency L behind the computation time.

06. FastFlow Framework & Stream Processing

This chapter introduces FastFlow, a C++ parallel programming framework developed primarily at the University of Pisa and the University of Torino. It implements algorithmic skeletons with a strong focus on stream processing and multi-core cache-coherent architectures.

1. FastFlow Design Philosophy

FastFlow is designed to overcome the performance bottlenecks associated with traditional lock-based synchronization (like `std::mutex` or `pthread_mutex_t`). It achieves high performance through:

- **Lock-Free and Wait-Free Queues:** The core communication mechanism relies on Single-Producer Single-Consumer (SPSC) lock-free queues.
- **Zero-Copy Routing:** Instead of copying large data payloads between pipeline stages or farm workers, FastFlow streams pointers to memory allocations.

- **Cache-Conscious Behavior:** By preventing threads from blocking and keeping them pinned to specific cores, FastFlow maximizes cache hotness and minimizes context-switching overhead.

---

2. Core Abstraction: `ff_node`

The fundamental building block in FastFlow is the node. A node represents an active entity (typically mapped to an OS thread) that executes a specific sequential computation.

2.1 The `svc` Method

To define a custom computation, you create a struct or class that inherits from `ff_node_t<IN, OUT>` (or the generic `ff_node`) and overrides the `svc` (service) method.

The `svc` method takes a pointer to an input task, processes it, and returns a pointer to the output task. * Returning a valid pointer pushes the task to the output queue. * Returning `GO_ON` instructs the runtime to fetch the next task without producing output for the current cycle. * Returning `EOS` (End Of Stream) signals the termination of the stream.

---

3. Implementing Skeletons

FastFlow provides container nodes that implement the standard algorithmic skeletons.

3.1 Pipeline (`ff_Pipe`)

A pipeline connects multiple `ff_node` instances sequentially. The output queue of stage i is exactly the input queue of stage $i + 1$. Because these are strictly 1-to-1 connections, the SPSC lock-free queues operate at maximum efficiency without requiring atomic Compare-And-Swap (CAS) operations, relying only on proper memory fences.

3.2 Farm (`ff_Farm`)

A farm distributes tasks among multiple worker nodes. * **Emitter:** Pulls from a Single-Producer Single-Consumer queue and pushes to N SPSC queues (one for each worker). * **Workers:** An array of `ff_node` instances executing the same `svc` logic. * **Collector:** Reads from N SPSC queues and merges the results into a single output stream.

---

4. C++ Implementation Example

Below is a complete example of a Pipeline containing a Farm, demonstrating how to set up the nodes and handle stream tokens.

```
#ifndef FASTFLOW_FARM_EXAMPLE_H
#define FASTFLOW_FARM_EXAMPLE_H

// Created by user

#include <iostream>
#include <vector>
#include <ff/ff.hpp>
#include <ff/farm.hpp>
#include <ff/pipeline.hpp>
```

```
using namespace ff;

// task definition
struct task_t {
    int id;
    double value;
};

// first stage: generates stream
struct generator : ff_node_t<task_t> {
    task_t* svc(task_t*) override {
        for (int i = 0; i < 100; ++i) {
            task_t* t = new task_t{i, i * 1.5};
            ff_send_out(t);
        }
        return EOS;
    }
};

// worker stage: processes stream in parallel
struct worker : ff_node_t<task_t> {
    task_t* svc(task_t* task) override {
        // simulate workload
        task->value = task->value * 2.0;
        return task;
    }
};

// final stage: collects results
struct collector : ff_node_t<task_t> {
    task_t* svc(task_t* task) override {
        std::cout << "PROCESSED TASK ID: " << task->id
                  << " | FINAL VALUE: " << task->value << "\n";
        delete task;
        return GO_ON;
    }
};

#endif // FASTFLOW_FARM_EXAMPLE_H

int main() {
    generator gen;
    collector col;

    // setup workers
    std::vector<std::unique_ptr<ff_node>> W;
    for(int i = 0; i < 4; ++i) {
        W.push_back(std::make_unique<worker>());
    }

    // build farm
    ff_Farm<task_t> farm(std::move(W));
    farm.remove_collector();
```

```
    // build pipeline: generator -> farm -> collector
    ff_Pipe<task_t> pipe(gen, farm, col);

    if (pipe.run_and_wait_end() < 0) {
        std::cerr << "FATAL: PIPELINE EXECUTION FAILED\n";
        return -1;
    }

    std::cout << "EXECUTION COMPLETED SUCCESSFULLY\n";
    return 0;
}
```

4.1 Memory Management Implications

Notice that the generator allocates memory on the heap (`new task_t`), and the collector is responsible for freeing it (`delete task`). Since FastFlow passes pointers, dynamic allocation is standard. However, frequent `new`/`delete` calls can cause contention on the glibc memory allocator. For extreme low-level performance, FastFlow applications often integrate custom thread-local memory allocators to bypass this OS-level bottleneck.

07. Data Parallelism & GPU Concurrency

This chapter extends the algorithmic skeleton framework to data-parallel workloads, exploring how to efficiently process large, partitioned data structures using both CPU-bound frameworks (FastFlow) and massively parallel architectures (GPUs via CUDA).

1. Data Parallelism in FastFlow

While stream parallelism processes discrete tokens over time, data parallelism operates on a finite, pre-existing data structure (like an array or matrix). FastFlow provides specific constructs to handle these workloads without the overhead of instantiating full pipeline or farm nodes for each element.

1.1 Parallel For (`ff::ParallelFor`)

The `ParallelFor` construct applies a function to a range of indices $[0, N)$. It splits the iteration space into chunks and assigns them to a pool of worker threads.

- **Chunking Strategy:** The runtime can distribute iterations statically (fixed size chunks) or dynamically (workers steal chunks when idle) to balance the load.
- **Low-Level Execution:** Unlike standard OpenMP, `ParallelFor` in FastFlow leverages the same lock-free worker pool underlying the `ff_Farm`. This avoids the OS-level thread creation and destruction overhead if multiple parallel loops are executed sequentially.

1.2 Parallel For with Reduction (`ff::ParallelForReduce`)

When the parallel loop must aggregate results into a single variable, synchronization is required to avoid data races. `ParallelForReduce` handles this by providing a thread-local accumulator and an associative reduction operator, eliminating the need for atomic instructions or mutexes during the map phase.

```
#ifndef FASTFLOW_DATAPARALLEL_EXAMPLE_H
#define FASTFLOW_DATAPARALLEL_EXAMPLE_H

#include <iostream>
#include <vector>
#include <thread>
```

```cpp
#include <ff/parallel_for.hpp>

using namespace ff;

void compute_data_parallel() {
    const size_t num_elements = 1000000;
    std::vector<double> data(num_elements, 2.0);
    double global_sum = 0.0;

    // initialize the parallel-for engine
    ParallelForReduce<double> pf(std::thread::hardware_concurrency());

    // execute map-reduce over the vector
    // chunking is handled automatically, using thread-local identity (0.0)
    pf.parallel_reduce(
        global_sum,
        0.0,
        0, num_elements,
        1, // step
        0, // auto chunk size
        [&](const long i, double& local_sum) {
            // map phase: square the element and accumulate locally
            local_sum += (data[i] * data[i]);
        },
        [](double& current_global, const double& local_result) {
            // reduce phase: merge local results into global
            current_global += local_result;
        }
    );

    std::cout << "PARALLEL REDUCTION COMPLETED. RESULT: " << global_sum << "\n";
}

#endif // FASTFLOW_DATAPARALLEL_EXAMPLE_H
```

---

2. GPU Concurrency & SIMT Architecture

When data parallelism scales to millions of independent operations, CPU thread counts (even highly optimized ones) become a bottleneck due to context switching and complex control units. GPUs solve this using a massively parallel execution model.

2.1 CPU vs GPU Architectures

To effectively program GPUs, one must understand their hardware design philosophy, which drastically differs from CPUs:

- **CPUs (Latency-Oriented):** Feature a few powerful cores, very large caches (L1/L2/L3), and sophisticated control units (branch prediction, out-of-order execution). They are optimized to execute a single thread as fast as possible.
- **GPUs (Throughput-Oriented):** Feature thousands of simpler cores, minimal control logic, and relatively small caches. They hide memory latency not through large caches, but through massive multi-threading, context-switching instantly when a thread blocks for memory.

2.2 The SIMT Execution Model

GPUs use the **SIMT (Single Instruction, Multiple Threads)** execution model, which is an evolution of CPU SIMD vectorization implementing a Single Program Multiple Data (SPMD) paradigm.

In CUDA, a parallel function executed on the GPU is called a **Kernel**. When a kernel is launched, it is executed by thousands of lightweight threads.

- **Threads** are grouped into **Blocks**. Threads within the same block execute concurrently on the same Streaming Multiprocessor (SM), can synchronize easily, and share fast on-chip memory.
- **Blocks** are grouped into a **Grid**. Blocks execute independently and cannot strictly synchronize with each other during kernel execution, allowing the GPU to scale the workload across any number of available SMs.

---

3. CUDA Basics and Synchronization

3.1 Memory Hierarchy

Effective C++ programming on GPUs requires strict, explicit control over pointer locations and memory spaces:

- **Global Memory:** The main VRAM. It is large but extremely slow (high latency). Accessible by all threads across all blocks.
- **Shared Memory:** A small, fast, user-managed cache allocated per Block. Used to share data among threads in the same block and avoid redundant global memory fetches.
- **Registers:** The fastest memory, strictly private to each individual thread.

3.2 CUDA Kernel Execution

Writing a CUDA kernel involves defining the C++ code that a single thread will execute. The thread computes its physical data partition by reading hardware-provided index variables.

```
#ifndef CUDA_VECTOR_ADD_H
#define CUDA_VECTOR_ADD_H

#include <iostream>
#include <cuda_runtime.h>

// kernel executed by each gpu thread
__global__ void vector_add(const float* a, const float* b, float* c, int n) {
    // compute global thread index mapping physical hardware to array index
    int i = blockDim.x * blockIdx.x + threadIdx.x;

    // bound check to avoid segmentation faults
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

void run_cuda_kernel() {
    int n = 100000;
    size_t bytes = n * sizeof(float);

    float *h_a, *h_b, *h_c;
```

```
    float *d_a, *d_b, *d_c;

    // allocate host memory (initialization omitted for brevity)
    h_a = (float*)malloc(bytes);
    h_b = (float*)malloc(bytes);
    h_c = (float*)malloc(bytes);

    // allocate device memory on the vram
    cudaMalloc(&d_a, bytes);
    cudaMalloc(&d_b, bytes);
    cudaMalloc(&d_c, bytes);

    // copy data from host ram to gpu global memory over pcie
    cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice);

    // define execution configuration
    int threads_per_block = 256;
    int blocks_per_grid = (n + threads_per_block - 1) / threads_per_block;

    // launch asynchronous kernel
    vector_add<<<blocks_per_grid, threads_per_block>>>(d_a, d_b, d_c, n);

    // explicit synchronization barrier
    // waits for the gpu to complete the kernel before cpu proceeds
    cudaDeviceSynchronize();

    // block cpu and copy result back to host
    cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost);

    std::cout << "CUDA KERNEL EXECUTED SUCCESSFULLY\n";

    // free memory to prevent leaks
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    free(h_a); free(h_b); free(h_c);
}

#endif // CUDA_VECTOR_ADD_H
```

3.3 Host-Device Synchronization and Latency Hiding

A critical aspect of GPU programming is managing the asynchronous nature of kernel launches. The `<<<...>>>` syntax queues the kernel on the device and returns control to the host CPU immediately.

To measure execution time accurately or ensure data is ready before subsequent CPU operations, explicit synchronization commands like `cudaDeviceSynchronize()` must be invoked. Without it, the host might attempt to read incomplete data from the GPU.

The overhead of PCI-Express memory transfers (`cudaMemcpy`) is massive and often represents the primary bottleneck. Advanced low-level optimization requires utilizing **CUDA Streams** to asynchronously overlap host-to-device memory copies with kernel execution, effectively hiding the PCIe latency.

08. Advanced GPU Concurrency & CUDA Optimizations

Building upon the basic SIMT (Single Instruction, Multiple Threads) execution model, this chapter explores the hardware-specific details necessary to achieve peak performance on GPUs. We focus on memory access patterns, the explicit management of the memory hierarchy, the mapping of software to hardware, and the overlapping of computation with data transfers.

1. Hardware Mapping: Software to Hardware

To heavily optimize CUDA kernels, the abstraction of Grids and Blocks must be mapped to the physical GPU components.

- **Grid → GPU:** The entire grid represents the full kernel execution on the GPU.
- **Block → Streaming Multiprocessor (SM):** Each block is assigned to a single SM. An SM can host multiple blocks concurrently, but a single block cannot be split across multiple SMs.
- **Thread → CUDA Core:** Individual threads execute on the cores within the SM.

2. The Warp Execution Model & Occupancy

While we program CUDA using a hierarchy of Grids and Blocks, the actual hardware execution on a Streaming Multiprocessor (SM) is managed in units called Warps.

- A warp consists of 32 contiguous threads from the same block.
- All threads in a warp execute the exact same instruction at the same time.
- **Warp Divergence:** If threads within a warp take different control-flow paths (e.g., due to an `if-else` statement), the SM must serialize the execution of the divergent paths. It executes the `true` path while masking the inactive threads, then executes the `false` path. This drastically reduces instruction throughput.

2.1 Occupancy and Latency Hiding

Unlike CPUs, which use large caches to reduce memory latency, GPUs use massive multi-threading to hide it. **Occupancy** is the ratio of active warps per SM to the maximum number of allowed warps. When a warp performs a high-latency operation (like a global memory fetch), the warp scheduler instantly switches to another "ready" warp. High occupancy ensures there are always ready warps, keeping the functional units busy. Occupancy is physically bounded by register usage per thread and shared memory usage per block.

3. Global Memory Coalescing

The bandwidth to global memory (VRAM) is a primary bottleneck in GPU computing. To maximize it, we must ensure coalesced memory accesses. When threads in a warp issue a memory load or store, the hardware attempts to group (coalesce) these requests into as few memory transactions as possible (typically 32, 64, or 128 bytes).

- **Coalesced Access:** Thread i accesses memory address $X + i$. The entire warp requests a contiguous block of memory, which can be fetched in a single transaction.
- **Uncoalesced Access:** Threads access scattered, non-contiguous addresses. The hardware must issue multiple memory transactions, wasting bandwidth.

3.1 Data Layout: AoS vs SoA

To promote coalesced accesses, data structures should often be refactored.

- **Array of Structures (AoS):** `struct { float x, y, z; } array[N];` Adjacent threads accessing `array[i].x` will fetch data spaced by the size of the struct, leading to uncoalesced accesses.
- **Structure of Arrays (SoA):** `struct { float x[N], y[N], z[N]; };` Adjacent threads accessing `x[i]` will fetch contiguous memory, achieving perfect coalescing.

4. Shared Memory and Tiling

Shared memory is a programmable L1 cache local to each SM. It is orders of magnitude faster than global memory but limited in size (e.g., 48KB or 96KB per block). We use shared memory to hold frequently accessed data. A common pattern is **Tiling**:

1. Threads in a block cooperatively load a "tile" of data from global memory into shared memory.
2. The block synchronizes to ensure all data is loaded.
3. Threads perform multiple computations using the fast shared memory.
4. The block synchronizes again, writes results to global memory, and moves to the next tile.

4.1 Tiling Implementation

```
#ifndef CUDA_MATRIX_MUL_TILED_H
#define CUDA_MATRIX_MUL_TILED_H


#include <iostream>
#include <cuda_runtime.h>

#define TILE_WIDTH 16

__global__ void matrix_mul_tiled(float* d_a, float* d_b, float* d_c, int width) {
    // allocate shared memory for tiles
    __shared__ float tile_a[TILE_WIDTH][TILE_WIDTH];
    __shared__ float tile_b[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x; int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    // identify the row and column of the output element to compute
    int row = by * TILE_WIDTH + ty;
    int col = bx * TILE_WIDTH + tx;

    float p_value = 0.0f;

    // loop over tiles
    for (int m = 0; m < width / TILE_WIDTH; ++m) {
        // load data into shared memory
        tile_a[ty][tx] = d_a[row * width + (m * TILE_WIDTH + tx)];
        tile_b[ty][tx] = d_b[(m * TILE_WIDTH + ty) * width + col];

        // sync threads to ensure tile is fully loaded
        __syncthreads();

        // compute partial dot product
        for (int k = 0; k < TILE_WIDTH; ++k) {
            p_value += tile_a[ty][k] * tile_b[k][tx];
        }

        // sync to prevent overwriting tiles in the next iteration
        __syncthreads();
    }

    d_c[row * width + col] = p_value;
```

```
}

#endif // CUDA_MATRIX_MUL_TILED_H
```

5. Advanced Synchronization: `__syncthreads()` and Beyond

While `__syncthreads()` provides a block-wide barrier, modern CUDA exposes warp-level primitives. **Warp-shuffle** instructions allow threads within the exact same warp to exchange data directly through registers, bypassing shared memory completely.

```
#ifndef CUDA_SHUFFLE_EXAMPLE_H
#define CUDA_SHUFFLE_EXAMPLE_H


#include <cuda_runtime.h>
#include <iostream>

__global__ void warp_reduce_kernel(int* d_data) {
    int val = d_data[threadIdx.x];

    // warp shuffle to read registers from other threads
    // sum all values within the warp (32 threads)
    for (int offset = 16; offset > 0; offset /= 2) {
        val += __shfl_down_sync(0xFFFFFFFF, val, offset);
    }

    // first thread writes the total sum
    if (threadIdx.x == 0) {
        d_data[0] = val;
    }
}

#endif // CUDA_SHUFFLE_EXAMPLE_H
```

6. Asynchronous Execution and CUDA Streams

To fully utilize both the CPU and the GPU, as well as the PCIe bus connecting them, we must overlap data transfers with kernel execution. This is achieved using CUDA Streams.

A stream is a sequence of commands (memory copies, kernel launches) that execute in order. Commands in different streams can execute concurrently.

6.1 Hiding PCIe Latency

By dividing the workload into chunks and assigning each chunk to a separate stream, the hardware can perform a `cudaMemcpyAsync` for chunk $i + 1$ while the GPU is executing the kernel for chunk i.

```
#ifndef CUDA_STREAMS_EXAMPLE_H
#define CUDA_STREAMS_EXAMPLE_H


#include <iostream>
#include <vector>
#include <cuda_runtime.h>

void run_with_streams(float* h_in, float* h_out, int n) {
```

```
    const int num_streams = 4;
    cudaStream_t streams[num_streams];

    // initialize streams
    for (int i = 0; i < num_streams; ++i) {
        cudaStreamCreate(&streams[i]);
    }

    int chunk_size = n / num_streams;
    size_t bytes = chunk_size * sizeof(float);

    float *d_in, *d_out;
    cudaMalloc(&d_in, n * sizeof(float));
    cudaMalloc(&d_out, n * sizeof(float));

    // queue operations asynchronously per stream
    for (int i = 0; i < num_streams; ++i) {
        int offset = i * chunk_size;

        // async copy host to device
        cudaMemcpyAsync(&d_in[offset], &h_in[offset], bytes, cudaMemcpyHostToDevice, streams[i]);

        // launch kernel on the specific stream
        // (assuming vector_add kernel exists)
        int threads = 256;
        int blocks = (chunk_size + threads - 1) / threads;
        // vector_add<<<blocks, threads, 0, streams[i]>>>(&d_in[offset], &d_out[offset], chunk_size);

        // async copy device to host
        cudaMemcpyAsync(&h_out[offset], &d_out[offset], bytes, cudaMemcpyDeviceToHost, streams[i]);
    }

    // wait for all streams to finish
    cudaDeviceSynchronize();

    std::cout << "ALL STREAMS EXECUTED SUCCESSFULLY. OVERLAPPING COMPLETE.\n";

    // clean up
    for (int i = 0; i < num_streams; ++i) {
        cudaStreamDestroy(streams[i]);
    }
    cudaFree(d_in);
    cudaFree(d_out);
}

#endif // CUDA_STREAMS_EXAMPLE_H
```

Note: To fully utilize `cudaMemcpyAsync`*, host memory must be page-locked (pinned memory) using* `cudaMallocHost` *rather than standard* `malloc` *or* `std::vector` *allocations.*

7. Arithmetic Intensity & The Memory Wall

A defining metric for low-level performance optimization is **Arithmetic Intensity**: the ratio of floating-point operations (FLOPs) to memory accesses (Bytes).

$$\text{Arithmetic Intensity} = \frac{\text{Operations}}{\text{Memory Traffic}}$$

- **Memory-Bound Kernels:** Low arithmetic intensity. Execution speed is gated by VRAM bandwidth.
- **Compute-Bound Kernels:** High arithmetic intensity. Execution speed is gated by the SM's clock speed and number of active CUDA cores.

The ultimate goal of tiling and register manipulation is to artificially increase the arithmetic intensity by reusing data directly on-chip, successfully bypassing the "Memory Wall" created by the relatively slow off-chip DRAM.

09. Communication Cost Models & Macro-Dataflow

This chapter formalizes the overhead of data movement in parallel architectures and introduces the Macro-Dataflow (MDF) execution model, a data-driven approach used to implement complex, irregular parallel graphs beyond simple algorithmic skeletons.

1. Communication Cost Models

When dealing with distributed memory (like MPI) or even complex cache-coherency traffic, communication time cannot be considered negligible. The time required to transmit a message of length L over a network or bus is typically modeled linearly:

$$T_{comm}(L) = t_{setup} + \frac{L}{B_w}$$

- t_{setup} **(Latency/Setup Time):** The constant overhead required to initiate the communication (e.g., protocol stack execution, network routing). It is independent of the message size.
- B_w **(Bandwidth):** The transmission rate of the channel (e.g., bytes per second).
- L **(Length):** The size of the payload being transmitted.

1.1 Impact on Farm Scalability

Recall the Farm skeleton service time $T_s = \max\left\{T_E, \frac{T_W}{n}, T_C\right\}$. With the communication cost model, the Emitter's execution time T_E is strictly bounded by the time it takes to send tasks to workers.

If the Emitter sends tasks of size L_{in}, its service time is at least the setup and transmission time: $T_E = t_{setup} + \frac{L_{in}}{B_w}$

Consequently, the maximum number of useful workers n_{max} is bottlenecked by the network:

$$n_{max} = \frac{T_W}{T_E} = \frac{T_W}{t_{setup} + \frac{L_{in}}{B_w}}$$

To increase n_{max}, we must either increase T_W (coarser grain size) or decrease communication overhead (e.g., sending pointers via shared memory as FastFlow does, where L is just 8 bytes and t_{setup} is an atomic queue insertion).

---

2. The Macro-Dataflow Execution Model

Traditional control-flow architectures rely on a Program Counter to fetch instructions sequentially. In a **Dataflow** architecture, there is no Program Counter; an instruction executes strictly when its input operands are available.

Macro-Dataflow (MDF) elevates this concept from the hardware instruction level to the function/task level. It allows the expression of generic Directed Acyclic Graphs (DAGs) of computation, where nodes are sequential functions and edges are data dependencies.

2.1 MDF Architecture Components

A typical runtime system supporting MDF consists of three main structures:

1. **Instruction Pool (or Program Memory):** Stores the static DAG. Each instruction knows its operation, its required number of input tokens, and the destination nodes for its output tokens.
2. **Matching Store:** A highly concurrent data structure that acts as a staging area. It collects incoming tokens and matches them to their destination instructions.
3. **Ready Queue:** When an instruction in the Matching Store receives all its required input tokens, it becomes "fireable" (ready) and is moved to the Ready Queue.
4. **Execution Units (Workers):** A pool of autonomous threads that constantly pull from the Ready Queue, execute the instruction, and push the resulting tokens back into the Matching Store.

2.2 Analytical Modeling of MDF

The performance of an MDF graph is determined by its **Critical Path**, which is the longest weighted path from the input nodes to the output nodes in the DAG. * **Parallelism Degree:** The average number of instructions that can be executed concurrently. * **Execution Time:** Lower bounded by the critical path weight W_{CP} (sum of T_w of nodes on the path) plus the communication costs along that path.

---

3. C++ Implementation Concepts

Implementing an efficient MDF runtime requires minimizing the contention on the Matching Store, which is heavily hit by concurrent workers.

```cpp
#include <vector>
#include <atomic>
#include <queue>
#include <mutex>
#include <condition_variable>

// forward declarations
struct token_t;
struct instruction_t;

// thread-safe ready queue
class ready_queue {
    std::queue<instruction_t*> q;
    std::mutex m;
    std::condition_variable cv;
public:
    void push(instruction_t* inst) {
        std::lock_guard<std::mutex> lk(m);
        q.push(inst);
        cv.notify_one();
    }

    instruction_t* pop() {
        std::unique_lock<std::mutex> lk(m);
        cv.wait(lk, [this]{ return !q.empty(); });
```

```
        auto inst = q.front();
        q.pop();
        return inst;
    }
};

struct instruction_t {
    int id;
    int expected_tokens;
    std::atomic<int> received_tokens{0};
    std::vector<int> dependent_ids;

    // pure business logic execution
    token_t* execute(std::vector<token_t*>& inputs);
};

// macro-dataflow worker loop
void mdf_worker(ready_queue& rq, class matching_store& ms) {
    while (true) {
        // fetch ready instruction (blocking)
        instruction_t* inst = rq.pop();
        if (!inst) break;

        // execute business logic (abstracted)
        std::vector<token_t*> inputs;
        token_t* result = inst->execute(inputs);

        // forward results to dependent instructions
        for (auto target_id : inst->dependent_ids) {
            // deliver_token handles the atomic increment and returns true
            // if the target instruction just reached its expected_tokens count
            if (ms.deliver_token(target_id, result)) {
                rq.push(ms.get_instruction(target_id));
            }
        }
    }
}
```

In advanced frameworks (like FastFlow's `ff_mdf` or Intel TBB's `flow::graph`), the Matching Store is often decentralized or implemented using lock-free data structures to prevent it from becoming the central bottleneck, thus preserving the theoretical parallel speedup of the DAG.

10. Distributed Skeletons Implementation

This chapter explores how to translate the high-level algorithmic skeletons (Pipeline and Farm) into functional distributed systems using the Message Passing Interface (MPI). In a distributed memory environment, the abstractions of shared queues and pointers are replaced by explicit network communications, requiring careful orchestration to avoid deadlocks and ensure load balancing.

1. Challenges in Distributed Skeletons

Implementing skeletons over MPI introduces complexities that are hidden in shared-memory frameworks like FastFlow: * **No Shared Pointers:** Data must be serialized and physically copied over the network. You cannot simply pass a pointer to a dynamically allocated object. * **Explicit Buffer Management:** The

programmer must allocate memory for sending and receiving messages. * **Deadlocks:** Cyclic dependencies in communication or mismatched send/receive pairs will cause the entire distributed application to hang permanently. * **Static Process Topology:** MPI requires launching a fixed number of processes at startup (`mpirun -np N`). Skeletons must be mapped onto these ranks statically.

---

2. Distributed Pipeline

A distributed pipeline maps each stage of the computation to one or more MPI ranks.

2.1 Static Mapping

If we have a 3-stage pipeline and 3 MPI ranks: * Rank 0 executes Stage 1. * Rank 1 executes Stage 2. * Rank 2 executes Stage 3.

Communication flows strictly from Rank i to Rank $i + 1$. * Rank 0 uses `MPI_Send` to Rank 1. * Rank 1 uses `MPI_Recv` from Rank 0, computes, and `MPI_Send` to Rank 2. * Rank 2 uses `MPI_Recv` from Rank 1.

2.2 Granularity and Bandwidth Optimization

To amortize the t_{setup} network latency, a distributed pipeline rarely sends individual small tokens. Instead, it aggregates data into larger chunks (batching). Rank 0 will pack multiple tasks into a single contiguous array and transmit it via a single `MPI_Send`.

---

3. Distributed Farm (Master-Worker)

The Farm pattern on distributed memory is universally known as the **Master-Worker** architecture. It is the most robust way to handle embarrassingly parallel workloads over a cluster.

3.1 Architecture

- **Master (Rank 0):** Acts as both Emitter and Collector. It maintains the global queue of tasks, dispatches them to available workers, and gathers the results.
- **Workers (Ranks 1 to N-1):** Infinite loops that wait for a message from the Master, execute the business logic, and send the result back.

3.2 Load Balancing Strategies

If tasks have highly variable execution times (irregular workload), a static distribution (e.g., using `MPI_Scatter` to give each worker K tasks upfront) leads to severe load imbalance. Some workers will finish early and sit idle while others are still computing.

On-Demand Scheduling (Dynamic Load Balancing): The Master sends exactly one task to each worker initially. Then, it waits for *any* worker to reply with a result. When a result is received, the Master immediately sends a new task to that specific worker. This inherently balances the load: faster nodes will automatically request and process more tasks than slower ones.

---

4. C++ Implementation of a Distributed Farm

Below is a complete implementation of a dynamically load-balanced Master-Worker architecture using MPI point-to-point communication.

```
#ifndef DISTRIBUTED_FARM_MPI_H
#define DISTRIBUTED_FARM_MPI_H

#include <iostream>
#include <vector>
#include <mpi.h>

// message tags for routing logic
#define TAG_TASK 1
#define TAG_RESULT 2
#define TAG_TERMINATE 3

void run_master(int num_workers, int total_tasks) {
    int tasks_sent = 0;
    int results_received = 0;

    // phase 1: initial distribution to fill the pipeline
    for (int i = 1; i <= num_workers && tasks_sent < total_tasks; ++i) {
        int task_data = tasks_sent++;
        MPI_Send(&task_data, 1, MPI_INT, i, TAG_TASK, MPI_COMM_WORLD);
        std::cout << "MASTER SENT TASK " << task_data << " TO WORKER " << i << "\n";
    }

    // phase 2: dynamic on-demand scheduling
    while (results_received < total_tasks) {
        int result;
        MPI_Status status;

        // wait for a result from any worker
        MPI_Recv(&result, 1, MPI_INT, MPI_ANY_SOURCE, TAG_RESULT, MPI_COMM_WORLD, &status);
        results_received++;

        std::cout << "MASTER RECEIVED RESULT " << result << " FROM WORKER " << status.MPI_SOURCE << "\n

        // if there are tasks left, send a new one to the worker that just finished
        if (tasks_sent < total_tasks) {
            int task_data = tasks_sent++;
            MPI_Send(&task_data, 1, MPI_INT, status.MPI_SOURCE, TAG_TASK, MPI_COMM_WORLD);
        }
    }

    // phase 3: broadcast termination signals
    for (int i = 1; i <= num_workers; ++i) {
        int dummy = 0;
        MPI_Send(&dummy, 1, MPI_INT, i, TAG_TERMINATE, MPI_COMM_WORLD);
    }
    std::cout << "MASTER SHUTTING DOWN SYSTEM\n";
}

void run_worker(int rank) {
    while (true) {
        int task_data;
        MPI_Status status;
```

```
        // block until a message arrives (either task or termination)
        MPI_Recv(&task_data, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);

        // break the infinite loop if master says we are done
        if (status.MPI_TAG == TAG_TERMINATE) {
            std::cout << "WORKER " << rank << " TERMINATING\n";
            break;
        }

        // execute workload
        int result = task_data * task_data;

        // send result back to master
        MPI_Send(&result, 1, MPI_INT, 0, TAG_RESULT, MPI_COMM_WORLD);
    }
}

// this would be called inside main() after MPI_Init
void mpi_farm_main() {
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int num_workers = size - 1;
    int total_tasks = 20;

    if (num_workers == 0) {
        std::cerr << "CRITICAL: NEED AT LEAST 2 PROCESSES FOR MASTER-WORKER\n";
        return;
    }

    if (rank == 0) {
        run_master(num_workers, total_tasks);
    } else {
        run_worker(rank);
    }
}

#endif // DISTRIBUTED_FARM_MPI_H
```

4.1 Optimization: Non-Blocking Master

In the basic implementation above, the Master is sequential and completely blocked during `MPI_Recv`. In a highly optimized system, the Master would use `MPI_Irecv` and `MPI_Isend` to manage thousands of workers concurrently without stalling, actively polling requests and processing other logic (like disk I/O) while waiting for network packets.

11. Advanced Patterns & Stateful Skeletons

This chapter explores complex parallel paradigms that go beyond basic stream processing and map-reduce data parallelism. We analyze patterns with internal dependencies, recursive structures, and stateful components, focusing on their theoretical models and implementation challenges.

1. Divide & Conquer

The Divide & Conquer (D&C) pattern models recursive algorithms where a problem is split into smaller sub-problems, solved independently, and then merged to form the final solution (e.g., MergeSort, QuickSort).

1.1 Structural Components

A D&C skeleton typically requires four functional parameters: * `is_base_case(x)`: Evaluates if the problem x is small enough to be solved sequentially. * `solve(x)`: The sequential solver for the base case. * `divide(x)`: Splits the problem into a set of sub-problems $\{x_1, x_2, \ldots, x_k\}$. * `merge(y_1, y_2, \dots, y_k)`: Combines the partial results into a final result.

1.2 Implementation Challenges

Naively spawning a new OS thread for every recursive `divide` call leads to exponential thread creation, crashing the system or destroying performance due to context-switching overhead.

- **Thread Pool & Task Stealing:** Modern runtimes (like Intel TBB or FastFlow's macro-dataflow engine) implement D&C by pushing the `divide` tasks into a shared pool. A fixed number of worker threads pull tasks from this pool.
- **Cut-off Depth:** To amortize the overhead of task creation, the recursion tree is truncated before reaching the absolute base case. When the depth reaches a certain threshold, the sub-tree is executed purely sequentially.

2. Stencil Pattern

The Stencil pattern is a data-parallel skeleton heavily used in scientific computing (e.g., Cellular Automata, Jacobi iteration for solving differential equations).

It operates on an N-dimensional grid. The value of a cell at time $t+1$ depends on its own value at time t and the values of its spatial neighbors at time t.

2.1 Ghost Cells and Halo Exchange

In a distributed memory architecture (e.g., using MPI), the global grid is partitioned into local sub-grids assigned to different nodes. However, computing the boundary cells of a local sub-grid requires data from the adjacent sub-grid on a different node.

- **Ghost Cells (Halos):** Each local grid is artificially expanded by adding a boundary layer (ghost cells).
- **Halo Exchange:** Before computing step $t+1$, neighboring nodes exchange their boundary data to update the ghost cells. This enforces a strict synchronization barrier and introduces communication overhead T_{comm} at every iteration.

3. Stateful Skeletons

Standard algorithmic skeletons assume functional purity: workers in a Farm or stages in a Pipeline operate exclusively on their current input token, without memory of past tokens.

However, many real-world applications require **Stateful Skeletons**, where a worker maintains an internal state updated by the stream of incoming data (e.g., accumulating statistics, maintaining a sliding window of network packets).

3.1 Concurrency Issues with State

If a Farm worker holds state, data routing becomes critical. * **Stateless Farm:** The Emitter can use a simple Round-Robin or On-Demand policy. Any task can go to any worker. * **Stateful Farm:** If the state is tied to a specific key in the data, the Emitter must implement a **Key-Based Routing** (or Hash-based

routing) policy. All tokens with the same key must be deterministically routed to the exact same worker to ensure they interact with the correct internal state without requiring distributed locks.

---

4. C++ Implementation of a Stateful Node

Using the FastFlow framework, implementing a stateful stage in a pipeline or farm requires defining member variables inside the `ff_node_t` derived struct.

```cpp
#ifndef STATEFUL_SKELETON_H
#define STATEFUL_SKELETON_H

#include <iostream>
#include <ff/ff.hpp>
#include <ff/pipeline.hpp>

using namespace ff;

// worker that maintains an internal state across multiple stream tokens
struct stateful_worker : ff_node_t<int> {
    int accumulator = 0;

    int* svc(int* task) override {
        // update the state safely since each worker thread has its own instance
        accumulator += *task;

        std::cout << "UPDATED INTERNAL STATE. CURRENT ACCUMULATOR: " << accumulator << "\n";

        // forward the updated state
        int* result = new int(accumulator);
        delete task;
        return result;
    }
};

#endif // STATEFUL_SKELETON_H
```

```
<div style="page-break-after: always;"></div>

# 12. Formal Semantics of Parallel Constructs

This chapter introduces the formal mathematical models used to describe the behavior of parallel skelet

## 1. Labeled Transition Systems (LTS)

To model parallel execution, we use Transition Systems. A Transition System is a tuple $\langle S, T, \
* $S$ is the set of possible states (configurations of our parallel program).
* $T$ is the set of terminal states.
* $\rightarrow \subseteq S \times S$ is the transition relation, denoting a single computational step.

For parallel skeletons, the state often includes the current active processes and the streams of data t

## 2. Semantics of Stream Parallel Skeletons
```

We model data streams as lists. Let $\langle x, \tau \rangle$ denote a stream where x is the head (cu

2.1 The `seq` Skeleton
The sequential skeleton applies a purely functional transformation f to a stream.

$$\langle \text{seq } f, \langle x, \tau \rangle \rangle \rightarrow \langle f(x) \rangle :: \langle \te

This rule states that the `seq` construct processes the head x, emits $f(x)$ to the output stream, an

2.2 The `pipe` Skeleton
A pipeline connects two skeletons Δ_1 and Δ_2. The output stream of Δ_1 becomes the

$$\frac{\langle \Delta_1, I \rangle \rightarrow O_{partial} :: \langle \Delta_1', I' \rangle \quad \lang

This inference rule specifies that a pipeline step is valid if the first stage processes the input to p

2.3 The `farm` Skeleton
A farm replicates a skeleton Δ over multiple parallel workers. The state of a farm must keep trac

Let $W_1 \dots W_n$ be instances of the worker Δ. A simplified transition for dispatching a task

$$\langle \text{farm}(\Delta), \langle x, \tau \rangle \rangle \rightarrow \langle W_k(x) \parallel \te

*(Note: $\parallel$ denotes concurrent execution. A formal LTS would include specific rules for the Coll

3. Semantic Equivalence and Rewriting Rules

By having a formal semantics, we can prove that two different skeleton compositions produce the exact sa

3.1 Normal Form
Every skeleton composition can be rewritten into a **Normal Form**, typically a single Farm wrapping a s

* **Rule 1 (Pipeline of Sequential nodes):**
  Two sequential nodes in a pipeline are semantically equivalent to a single sequential node applying
  $$\text{pipe}(\text{seq } f, \text{seq } g) \equiv \text{seq } (g \circ f)$$

* **Rule 2 (Farm of Farms):**
  A farm whose workers are farms is equivalent to a single farm with a flattened worker pool.
  $$\text{farm}(\text{farm}(\Delta)) \equiv \text{farm}(\Delta)$$

* **Rule 3 (Farm Expansion):**
  A sequential node can always be trivially parallelized by a farm (assuming stateless functional beha
  $$\text{seq } f \equiv \text{farm}(\text{seq } f)$$

4. Bisimulation

When dealing with non-determinism (e.g., in the `farm` Collector), basic equivalence is not enough. We u

Two systems P and Q are bisimilar if:
1. Every observable transition $P \xrightarrow{\alpha} P'$ can be matched by $Q \xrightarrow{\alpha} Q'$
2. Vice versa.

This ensures that rewriting a `pipe` into a `farm` (or vice-versa, when optimizing) does not introduce a

13. Autonomic Computing & Quality of Service (QoS)

This chapter addresses the dynamic nature of modern execution environments. Workloads are rarely perfec

1. The Need for Dynamic Adaptability

Standard algorithmic skeletons (like a basic `ff_Farm`) are usually instantiated with a static parallel
If the workload is irregular, a statically configured farm suffers from two potential issues:
1. **Under-provisioning:** If the task difficulty increases, the Service Time (T_s) degrades, violati
2. **Over-provisioning:** If the task difficulty decreases, workers finish quickly and idle, wasting CP

To solve this, the framework must adjust the parallelism degree n at runtime.

---

2. The MAPE Control Loop

Autonomic systems are governed by a continuous feedback loop known as the **MAPE** loop:

* **Monitor:** Sensors collect low-level metrics from the system without introducing significant overhe
* **Analyze:** The system compares the monitored metrics against the target QoS constraint (e.g., "main
* **Plan:** A heuristic or analytical model calculates the optimal system configuration. If T_s is to
* **Execute:** The runtime dynamically reconfigures the graph. It spawns new threads to increase n, o

---

3. Analytical Modeling for the Planning Phase

The Planning phase relies on performance models to make correct decisions without oscillating or guessi

Let the target service time be T_{ideal}.
The current service time is measured as T_{curr}, and the current number of workers is n_{curr}.
Assuming the Emitter and Collector are not the bottleneck, the worker service time is $T_W \approx T_{c

To find the new optimal number of workers n_{opt} required to achieve T_{ideal}:
$$n_{opt} = \left\lceil \frac{T_W}{T_{ideal}} \right\rceil = \left\lceil \frac{T_{curr} \times n_{curr}

3.1 Avoiding Thrashing
Dynamically adding and removing threads has a cost (OS context allocation, cache warming). The Autonomi

---

4. C++ Implementation: Dynamic Worker Pool

Implementing a dynamic farm requires a carefully designed execution loop where workers can be cleanly r

```cpp
#ifndef AUTONOMIC_FARM_H
#define AUTONOMIC_FARM_H
```

```
#include <iostream>
#include <vector>
#include <thread>
#include <atomic>
#include <queue>
#include <mutex>
#include <condition_variable>

struct task_t {
    int id;
    long complexity;
};

class dynamic_pool {
    std::vector<std::thread> workers;
    std::queue<task_t> tasks;
    std::mutex mtx;
    std::condition_variable cv;

    std::atomic<bool> stop_flag{false};
    std::atomic<int> active_workers{0};
    std::atomic<int> target_workers{0};

    // worker main loop
    void worker_loop() {
        while (true) {
            task_t current_task;
            {
                std::unique_lock<std::mutex> lock(mtx);
                cv.wait(lock, [this] {
                    return !tasks.empty() || stop_flag || active_workers > target_workers;
                });

                // graceful self-termination if the pool is scaling down
                if (active_workers > target_workers) {
                    active_workers--;
                    std::cout << "WORKER SCALING DOWN. THREAD EXITING.\n";
                    return;
                }

                if (stop_flag && tasks.empty()) return;

                current_task = tasks.front();
                tasks.pop();
            }

            // execute business logic
            // ... computation here ...
        }
    }

public:
    dynamic_pool(int initial_size) {
```

```
        set_parallelism_degree(initial_size);
    }

    // interface for the autonomic manager to adjust size at runtime
    void set_parallelism_degree(int n) {
        target_workers = n;
        std::lock_guard<std::mutex> lock(mtx);

        // scale up
        while (active_workers < target_workers) {
            active_workers++;
            workers.emplace_back(&dynamic_pool::worker_loop, this);
            std::cout << "WORKER SCALING UP. NEW THREAD SPAWNED.\n";
        }

        // notify all in case some need to scale down
        cv.notify_all();
    }

    void submit(task_t t) {
        std::lock_guard<std::mutex> lock(mtx);
        tasks.push(t);
        cv.notify_one();
    }

    void shutdown() {
        stop_flag = true;
        cv.notify_all();
        for (auto& w : workers) {
            if (w.joinable()) w.join();
        }
    }
};

#endif // AUTONOMIC_FARM_H
```

Advanced frameworks like FastFlow provide built-in autonomic nodes (`ff_aFarm`) that encapsulate the MAPE loop, abstracting away the thread orchestration and relying strictly on lock-free queues even during dynamic resizing.

14. SIMD & Vectorization on CPUs

This chapter explores Single Instruction, Multiple Data (SIMD) programming on modern CPUs. It bridges the gap between purely sequential code and massively parallel GPU execution by exploiting data parallelism directly within the CPU cores.

1. Vector Units and Registers

Modern CPU cores contain dedicated vector units capable of processing multiple data elements concurrently using wide registers.

- **Scalar Execution:** Traditional ALUs process one instruction on one piece of data at a time.
- **SIMD Execution:** A single instruction (e.g., an addition) is applied simultaneously to multiple data elements packed inside a large SIMD register.
- **Register Widths:**

- **SSE:** 128-bit registers (fits 4 standard 32-bit floats).
- **AVX/AVX2:** 256-bit registers (fits 8 floats).
- **AVX-512:** 512-bit registers (fits 16 floats).

2. Automatic Compiler Vectorization

The easiest way to leverage SIMD is to rely on the compiler's auto-vectorizer. The compiler analyzes loops and translates them into SIMD instructions automatically.

2.1 Loop Unrolling

To help the compiler pipeline instructions and hide latencies, developers can force loop unrolling using pragmas. Unrolling reduces control instruction overhead (branching, index updates) and exposes more independent operations (Instruction Level Parallelism, ILP).

```
#ifndef SIMD_AUTO_VECT_H
#define SIMD_AUTO_VECT_H

#include <iostream>
#include <algorithm>
#include <vector>
#include <cmath>

void compute_max_unrolled(const std::vector<float>& data) {
    float max_0 = -INFINITY, max_1 = -INFINITY;
    float max_2 = -INFINITY, max_3 = -INFINITY;

    // hint the compiler to unroll the loop
    #pragma GCC unroll 4
    for (size_t i = 0; i < data.size(); i += 4) {
        // independent max operations prevent read-after-write (raw) dependencies
        max_0 = std::max(max_0, data[i]);
        max_1 = std::max(max_1, data[i+1]);
        max_2 = std::max(max_2, data[i+2]);
        max_3 = std::max(max_3, data[i+3]);
    }

    float final_max = std::max({max_0, max_1, max_2, max_3});
    std::cout << "MAX VALUE FOUND: " << final_max << "\n";
}

#endif // SIMD_AUTO_VECT_H
```

Note: Excessive unrolling can cause register spilling, which hurts performance by forcing data back to the L1 cache.

3. Explicit Vectorization with Intrinsics

When compilers fail to auto-vectorize complex logic, we use **Intrinsics**. Intrinsics are C/C++ functions mapped directly to assembly SIMD instructions, giving explicit, low-level control over the vector units.

```
#ifndef SIMD_INTRINSICS_H
#define SIMD_INTRINSICS_H

#include <iostream>
#include <immintrin.h> // required for avx intrinsics
```

```
void add_arrays_avx(const float* a, const float* b, float* result, size_t size) {
    size_t i = 0;

    // process 8 floats (256 bits) at a time
    for (; i + 7 < size; i += 8) {
        // load 256 bits of data from unaligned memory
        __m256 vec_a = _mm256_loadu_ps(&a[i]);
        __m256 vec_b = _mm256_loadu_ps(&b[i]);

        // perform parallel addition
        __m256 vec_res = _mm256_add_ps(vec_a, vec_b);

        // store results back
        _mm256_storeu_ps(&result[i], vec_res);
    }

    // handle remaining elements sequentially (tail case)
    for (; i < size; ++i) {
        result[i] = a[i] + b[i];
    }

    std::cout << "AVX ADDITION COMPLETED\n";
}

#endif // SIMD_INTRINSICS_H
```

By explicitly loading data into `__m256` types, we bypass the compiler's conservative dependency analysis and force the hardware to process the payload at maximum throughput.

15. Types of Parallelism & Advanced Scalability

This chapter formalizes the three primary forms of parallelism and delves into the analytical implications of parallel overheads, including the phenomenon of super-linear speedup.

1. The Three Pillars of Parallelism

Parallel execution models can be broadly categorized into three families, depending on how data and control flow are managed.

1.1 Data Parallelism

Data parallelism involves applying the same operation concurrently to elements of a data collection (arrays, matrices, trees). * **Input/Output:** A collection of data goes in, and a collection of data comes out. * **Characteristics:** Highly synchronous. Operations on different elements are assumed to be independent. It is the core paradigm exploited by GPUs (SIMT) and CPU vector units (SIMD).

1.2 Stream Parallelism

Stream parallelism operates on a continuous sequence (stream) of data items. The computation is structured as a sequence of stages, each applying a specific transformation. * **Input/Output:** A stream of tokens goes in, a stream of tokens comes out. * **Characteristics:** The state of one stage is isolated from the others. It exploits pipeline concurrency and is ideal for systems where the total input size is unknown or infinite.

1.3 Task Parallelism

Task parallelism models activities as a pool of independent, heterogeneous tasks. There is no strict ordering or single shared data structure. * **Input/Output:** Discrete tasks are submitted, and discrete results are produced asynchronously. * **Characteristics:** Execution relies on a "greedy" scheduling mechanism. Workers (threads/processes) pick the next available task from a shared pool.

```
#ifndef TASK_PARALLELISM_POOL_H
#define TASK_PARALLELISM_POOL_H

#include <iostream>
#include <vector>
#include <future>
#include <numeric>

// simulate heterogeneous tasks
int compute_heavy_task(int x) { return x * x; }
int compute_light_task(int x) { return x + 1; }

void execute_task_pool() {
    std::vector<std::future<int>> results;

    // tasks are dispatched asynchronously to an underlying thread pool
    for (int i = 0; i < 10; ++i) {
        if (i % 2 == 0) {
            results.push_back(std::async(std::launch::async, compute_heavy_task, i));
        } else {
            results.push_back(std::async(std::launch::async, compute_light_task, i));
        }
    }

    // gather results
    int total = 0;
    for (auto& f : results) {
        total += f.get();
    }

    std::cout << "all tasks completed. total: " << total << "\n";
}

#endif // TASK_PARALLELISM_POOL_H
```

---

2. Analytical Overheads in Parallel execution

Parallelism is never free. The parallelization process introduces specific overheads that do not exist in the sequential version of the algorithm.

2.1 Pipeline Data Movement Overhead (T_{move})

When analyzing a pipeline, data must be transferred between stages (e.g., passing a pointer via a queue, or sending a payload over MPI). Let T_{task} be the computation time and T_{move} be the communication time.

The service time of the pipeline is lower-bounded by the maximum of these two:

$$T_c^{pipeline} \approx \max(T_{task}, T_{move})$$

If $T_{move} \geq T_{task}$, the pipeline yields no speedup. The system becomes completely bottlenecked by the data movement, highlighting the necessity of coarse-grained computation.

2.2 Task Scheduling Overhead ($\Delta(k)$)

In a Task Parallel model with N total tasks and k workers, let W be the total sequential work.

The parallel completion time T_c^{task} is modeled as:

$$T_c^{task}(N, k) \geq \max\left(\frac{W + \Delta(k)}{k}, T_{task} + T_a\right)$$

- $\Delta(k)$ represents the overhead of task scheduling and synchronization (e.g., mutex contention on the shared task queue).
- As the number of workers k increases, $\Delta(k)$ typically grows. If the tasks are too small (fine-grained), $\Delta(k)$ will dominate W, destroying efficiency.

---

3. Super-Linear Speedup

Ideally, the maximum speedup on p processors is p (linear speedup). However, in rare but important circumstances, a parallel algorithm can achieve a speedup strictly greater than p:

$$S(p) > p$$

3.1 Causes of Super-Linear Speedup

1. **Cache Effects (The Working Set):** In a sequential execution, a massive dataset might not fit into the CPU's L1/L2 caches, causing frequent, high-latency DRAM fetches (cache thrashing). When the problem is partitioned across p cores, the smaller data chunks may now fit perfectly into the individual L1/L2 caches of each core. The sudden elimination of main memory latency leads to a performance jump greater than the raw increase in computational power.
2. **Search Algorithms:** In algorithms traversing search trees (like Branch & Bound or parallel DFS), a parallel thread might quickly stumble upon the optimal solution or a cutoff condition by exploring a specific branch, drastically reducing the total number of nodes visited compared to the sequential execution path.

16. Modern C++ Essentials for Parallelism

This chapter bridges the theoretical parallel paradigms with their practical implementation in Modern C++ (C++20). High-performance concurrent programming requires precise control over memory, types, and compiler optimizations.

1. Type Deduction and Initialization

Modern C++ introduces mechanisms to let the compiler deduce types statically, reducing boilerplate but introducing low-level nuances that can impact performance or correctness.

1.1 `auto` vs `decltype`

The `auto` keyword deduces the type from the right-hand side of an assignment. However, **`auto` drops top-level `const` qualifiers and references (`&`)**.

To preserve them, one must explicitly write `const auto&` or use `decltype`. `decltype(expr)` inspects the declared type of an expression without evaluating it, strictly preserving both `const` and reference semantics. This is critical in generic template programming where the return type of a parallel operation might be a complex reference.

1.2 Uniform Initialization Pitfalls

C++ provides multiple ways to initialize objects, but using parentheses `()` versus curly braces `{}` (initializer lists) can dramatically change the semantics, especially in containers.

- `std::vector<int> v(3, 1);` invokes the constructor to create a vector of 3 elements, all initialized to the value `1`.
- `std::vector<int> v{3, 1};` invokes the initializer list constructor, creating a vector of exactly 2 elements: `3` and `1`.

2. Memory Semantics: References and Pointers

In high-performance computing, deep copying large data structures (like million-element vectors) destroys performance and wastes memory bandwidth.

- **Aliases over Copies:** C++ uses references (`&`) to create aliases to existing storage. Unlike raw C pointers, references cannot be null and do not require dereferencing syntax, making the code safer and cleaner.
- **`const` References:** Passing data as `const Type&` guarantees no deep copy is made and prevents modification. More importantly, it signals to the compiler that the data is read-only, enabling aggressive optimization passes (e.g., caching values in registers without fear of unexpected mutations).

3. The Standard Template Library (STL) Architecture

The STL is the core of modern C++ and is built on three cooperating pillars:

1. **Containers:** Data structures that manage storage (e.g., `std::vector`, `std::list`).
2. **Iterators:** Pointer-like objects (`begin()`, `end()`) that provide a uniform interface to traverse any container, decoupling the data structure from the algorithm.
3. **Algorithms:** Generic functions (`std::sort`, `std::transform`, `std::find_if`) that operate on ranges defined by iterators.

3.1 Parallel STL and Execution Policies

Starting from C++17 and expanded in C++20, many STL algorithms accept an **Execution Policy** parameter to automatically parallelize their workload (Data Parallelism).

- `std::execution::seq`: Strictly sequential.
- `std::execution::par`: Parallel execution across multiple threads.
- `std::execution::par_unseq`: Parallel and vectorized (SIMD).

The Silent Fallback Issue: The parallel STL algorithms often rely on external threading backends (like Intel TBB - Threading Building Blocks). If the backend is not installed or linked correctly during compilation, the compiler will silently fall back to the sequential implementation. The code will compile and run correctly, but yield zero parallel speedup.

4. Implementation Example

```cpp
#ifndef MODERN_CPP_ESSENTIALS_H
#define MODERN_CPP_ESSENTIALS_H

#include <iostream>
#include <vector>
#include <algorithm>
#include <execution>

void demonstrate_cpp_features() {
    // uniform initialization difference
    std::vector<int> count_initialized(1000000, 1);
    std::vector<int> list_initialized{1000000, 1};

    // type deduction semantics
    const int val = 42;
    auto copy_val = val;             // int
    decltype(val) strict_val = val;  // const int

    // parallel sorting using stl execution policies
    // requires intel tbb to actually spawn threads
    std::sort(std::execution::par, count_initialized.begin(), count_initialized.end());

    std::cout << "CPP FEATURES EXECUTED AND SORT COMPLETED\n";
}

#endif // MODERN_CPP_ESSENTIALS_H

<div style="page-break-after: always;"></div>

# 17. Advanced C++ Semantics

This chapter delves into advanced C++ abstractions that are heavily utilized in modern parallel programm

## 1. Function Objects (Functors)

A Functor is a C++ class or struct that overloads the function call operator `operator()`. This allows a

* **Internal State:** Unlike a plain function, a functor can maintain an internal state across multiple
* **Optimization:** Because the type of the functor is explicitly known at compile time, the compiler ca

```cpp
#ifndef FUNCTOR_EXAMPLE_H
#define FUNCTOR_EXAMPLE_H

#include <iostream>
#include <vector>
#include <algorithm>

// A functor maintaining internal state
struct MaxTracker {
    int current_max = -999999; // Internal state
```
```

```
    void operator()(int element) {
        if (element > current_max) {
            current_max = element;
        }
    }
};

void run_functor() {
    std::vector<int> data = {1, 5, 2, 8, 3};
    MaxTracker tracker;

    // We must capture the functor by reference using std::ref,
    // otherwise std::for_each will copy it and discard the internal state update
    std::for_each(data.begin(), data.end(), std::ref(tracker));

    std::cout << "Max found by functor: " << tracker.current_max << "\n";
}

#endif // FUNCTOR_EXAMPLE_H
```

2. Lambda Expressions

Introduced in C++11, lambda expressions provide a concise syntax for defining anonymous function objects inline. They are the backbone of task-based parallelism.

2.1 The Capture Clause `[]`

The capture clause defines the "environment" or closure of the lambda, specifying which external variables the lambda can access.

- `[=]`: Capture everything by value (creates a local copy). By default, these copies are `const`. To modify the local copy without affecting the original variable, the `mutable` keyword must be added.
- `[&]`: Capture everything by reference (modifies the original variable).
- `[x, &y]`: Capture `x` by value and `y` by reference.

2.2 `std::function` and Type Erasure

Every lambda expression has a unique, anonymous type generated by the compiler. This makes it impossible to store different lambdas (even with the same signature) in a standard container like `std::vector` without type erasure.

`std::function<ReturnType(Args...)>` provides this type erasure. It can store plain functions, functors, and lambdas with matching signatures in a unified type, allowing for heterogeneous task pools. However, this flexibility incurs a slight runtime overhead compared to calling a lambda directly.

3. Resource Acquisition Is Initialization (RAII)

RAII is a core C++ idiom that binds the lifecycle of a resource (memory, file handles, network sockets, hardware locks) to the lifetime of an object on the stack.

- **Acquisition:** The resource is acquired in the object's constructor.
- **Release:** The resource is released in the object's destructor.

Because C++ guarantees that destructors of stack-allocated objects are called when they go out of scope (even if an exception is thrown or a `return` statement is executed early), RAII guarantees leak-free resource management.

3.1 RAII in Concurrency

RAII is the safest way to manage mutual exclusion. Instead of manually calling `mtx.lock()` and `mtx.unlock()`—which is prone to deadlocks if the programmer forgets to unlock—we use `std::lock_guard`.

```
#include <mutex>
#include <vector>

std::mutex shared_mtx;
std::vector<int> shared_data;

void safe_insert(int value) {
    // Lock is acquired by the constructor of lock_guard
    std::lock_guard<std::mutex> lock(shared_mtx);

    shared_data.push_back(value);

    // Lock is automatically released here when 'lock' goes out of scope,
    // guaranteeing no deadlocks even if push_back throws an exception.
}
```

---

18. Move Semantics and Perfect Forwarding

This chapter explores C++11 Move Semantics, a pivotal optimization for high-performance computing that eliminates unnecessary deep copies of large objects by safely "stealing" their underlying resources.

1. L-values and R-values

To understand move semantics, one must distinguish between value categories:

- **L-value:** An object that occupies an identifiable location in memory (it has a name/address). Example: `std::vector<int> v;`.
- **R-value:** A temporary object that does not have a persistent memory address. Example: `std::vector<int>(1000, 1)` or the result of a mathematical expression `x + y`.

2. Move Semantics (`std::move`)

Deep copying large containers degrades performance. If the source of a copy is an r-value (a temporary object about to be destroyed anyway), copying its data is wasteful. We should instead **move** (transfer ownership of) the underlying pointers.

2.1 R-value References `&&`

An r-value reference (denoted by `&&`) binds exclusively to temporary objects or objects explicitly cast to r-values.

2.2 `std::move`

The `std::move` function **does not move anything**. It merely performs a static cast, converting an l-value into an r-value reference (`Type&&`), marking it as eligible to have its resources stolen.

```
std::string s1 = "Heavy Payload";
// s2 steals the internal char* pointer of s1.
```

```
// s1 is left in a valid but empty/unspecified state.
std::string s2 = std::move(s1);
```

Warning: If `std::move` is applied to a `const` object, the move silently degrades into a standard copy, because moving requires modifying the source object to leave it in an empty state.

3. The Rule of Five and Rule of Zero

If you are implementing a custom low-level data structure managing raw pointers (like a custom memory pool), you must define the **Rule of Five** to ensure proper resource management and enable move semantics:

1. Destructor
2. Copy Constructor
3. Copy Assignment Operator
4. **Move Constructor**
5. **Move Assignment Operator**

The Rule of Zero: If your class only contains STL containers (`std::vector`, `std::string`) or smart pointers, the compiler will automatically generate highly optimized versions of all five methods for you. You should write zero custom memory management code.

4. Perfect Forwarding (`std::forward`)

In generic template programming, when a function receives a parameter, the parameter has a name, making it an l-value inside the function body—even if it was originally passed as an r-value.

If we want to pass this parameter to another nested function while perfectly preserving its original value category (l-value stays l-value, r-value stays r-value), we use **Forwarding References** (`T&&` in a template context) and `std::forward`.

```
template <typename T>
void wrapper(T&& arg) { // T&& here is a forwarding reference, not just an r-value reference
    // std::forward conditionally casts 'arg' to an r-value ONLY IF
    // the original argument passed to wrapper was an r-value.
    process(std::forward<T>(arg));
}
```

This ensures that we don't accidentally copy large objects that could have been moved, achieving "Perfect Forwarding".

18. Practical C++ Concurrency & Thread Management

This chapter bridges the gap between theoretical parallel paradigms (like Map) and their practical implementation using Modern C++ features. It focuses on thread lifecycle management, data partitioning, safe argument passing, and fundamental synchronization mechanisms.

1. Thread Lifecycle and Management

In C++, parallel execution is primarily managed through the `std::thread` class. Threads share the same memory space (heap and global variables) but maintain private execution stacks.

1.1 Spawning, Joining, and Detaching

When a `std::thread` object is instantiated, the runtime immediately spawns an OS-level thread to execute the provided callable object (a function, functor, or lambda).

- **Join (`.join()`):** Blocks the calling thread until the spawned thread completes its execution. This is the primary synchronization point for embarrassingly parallel patterns like Map.
- **Detach (`.detach()`):** Separates the thread of execution from the `std::thread` object, allowing execution to continue independently.
- **The Termination Pitfall:** If a `std::thread` object goes out of scope and is destroyed while still "joinable" (i.e., neither joined nor detached), the C++ runtime will invoke `std::terminate()`, crashing the program.

1.2 Threads are Move-Only

A `std::thread` represents a unique hardware resource and cannot be copied. Its copy constructor and copy assignment operator are explicitly deleted (`= delete`). To manage collections of threads efficiently, we use containers like `std::vector` combined with move semantics: * Avoid `push_back(std::thread(...))` if it creates unnecessary temporaries. * Prefer `emplace_back(...)` to construct the thread directly in the memory of the container.

---

2. Implementing a Parallel Map and Memory Optimization

A Data Parallel Map involves splitting an input collection, assigning disjoint ranges to threads, and gathering the output.

2.1 The `reserve()` Optimization

When dynamically populating a `std::vector` of threads (or results), repeatedly calling `emplace_back` can trigger multiple background memory reallocations if the vector's capacity is exceeded. By calling `vector.reserve(num_threads)` upfront, the exact required contiguous memory is allocated exactly once on the heap, drastically reducing allocation overhead and preventing cache invalidation.

2.2 Hardware Concurrency and Oversubscription

Spawning threads is not free. * **Compute-Bound Tasks:** If threads perform heavy CPU computations (like mathematical transformations), spawning more threads than the available physical cores (oversubscription) leads to negative scaling. The threads fight for CPU time, incurring massive context-switching overhead. * **I/O-Bound Tasks:** If threads frequently block (e.g., waiting for disk or network), oversubscription can be beneficial, as the OS can schedule another thread while one is blocked.

---

3. Argument Passing and Closure Pitfalls

Passing arguments to threads—especially when using lambdas in a loop—requires strict attention to value categories to avoid catastrophic race conditions.

3.1 Passing by Reference (`std::ref`)

By default, `std::thread` copies or moves arguments into the internal storage of the new thread. To explicitly pass an argument by reference (e.g., a shared output vector), you must wrap it in `std::ref()`.

3.2 The Lambda Capture Trap

Consider a loop spawning threads, where `id` is the loop index:

```
// DANGEROUS: Capturing the loop index by reference
for (int id = 0; id < num_threads; ++id) {
    threads.emplace_back([&]() {
```

```
        results[id] = compute(id); // RACE CONDITION!
    });
}
```

If you capture the environment entirely by reference (`[&]`), the spawned thread accesses the memory location of the loop variable `id`. Because the main thread continues executing the loop rapidly, the value of `id` will change (or go out of scope) *before* the spawned thread reads it. Multiple threads might end up reading the exact same wrong `id`.

The Fix: Always capture loop variables and primitive parameters by value:

```
// SAFE: Capture 'id' by value, and 'results' by reference
for (int id = 0; id < num_threads; ++id) {
    threads.emplace_back([id, &results]() {
        results[id] = compute(id);
    });
}
```

---

4. Mutual Exclusion and RAII

When threads must access shared mutable state (e.g., a shared counter or queue), we use `std::mutex` to enforce critical sections. However, manually calling `mtx.lock()` and `mtx.unlock()` is error-prone; if an exception occurs inside the critical section, the unlock may be skipped, causing a permanent deadlock.

C++ solves this using RAII (Resource Acquisition Is Initialization) wrappers:

1. **`std::lock_guard`:** The simplest and most efficient wrapper. It acquires the mutex on construction and releases it upon destruction (when it goes out of scope).
2. **`std::unique_lock`:** A more flexible wrapper. It allows deferred locking, explicit `.unlock()`, and transferring ownership (movable). It incurs a slight overhead compared to `lock_guard` but is mandatory for Condition Variables.
3. **`std::scoped_lock`:** Used to acquire multiple mutexes simultaneously without deadlocks, utilizing a safe lock-ordering algorithm under the hood.

---

5. Condition Variables and Thread Synchronization

A `std::condition_variable` is used when a thread needs to sleep and wait for a specific condition to become true (e.g., waiting for a Producer to put data into an empty buffer).

5.1 The Wait Pattern

A thread waiting on a condition variable must hold a `std::unique_lock`. To prevent "spurious wakeups" (where the OS wakes the thread without the condition being met), the `wait` method should be provided with a predicate lambda:

```
std::unique_lock<std::mutex> lock(mtx);
// The thread atomically releases the lock and goes to sleep.
// When woken up, it re-acquires the lock and checks the lambda condition.
cv.wait(lock, []{ return !buffer.empty(); });
```

5.2 Signaling (`notify_one` vs `notify_all`)

When a thread changes the shared state (e.g., adds an item to the buffer), it must signal the condition variable.

- **`notify_one()`:** Wakes up exactly one waiting thread. Used when only one thread can consume the new resource. It is highly efficient. The OS decides which thread to wake up (no FIFO guarantee).
- **`notify_all()`:** Wakes up all waiting threads. Used when the state change affects everyone (e.g., a global termination flag).

5.3 Optimization: Notify Outside the Lock

A common performance optimization is to call `notify` *after* releasing the mutex:

```
{
    std::lock_guard<std::mutex> lock(mtx);
    buffer.push(data);
} // Lock is automatically released here

cv.notify_one(); // Notify outside the critical section
```

If `notify_one()` is called inside the critical section, the woken thread will immediately try to acquire the mutex. Finding it still locked by the signaling thread, it will instantly be forced back to sleep, wasting CPU cycles in an unnecessary context switch. Notifying outside the lock prevents this contention.

19. Advanced Concurrency Patterns

This chapter moves beyond basic thread spawning and explores sophisticated concurrent programming paradigms. We examine classical synchronization problems, one-shot communication mechanisms, and the architecture of a generic Thread Pool for optimal resource management.

1. The Producer-Consumer Problem

The Producer-Consumer pattern is a classic synchronization problem where one or more "Producers" generate data and place it into a shared buffer, while one or more "Consumers" extract and process that data.

1.1 Unbounded vs. Bounded Buffers

- **Unbounded Buffer:** The queue (e.g., `std::deque`) can grow infinitely. Producers never block; they only acquire the mutex to insert data and then notify consumers.
- **Bounded Buffer:** The queue has a maximum capacity. Producers must wait if the buffer is full, requiring two separate condition variables: `cv_empty` (for consumers to wait) and `cv_full` (for producers to wait).

1.2 Condition Variables and "Mesa Semantics"

When a thread is awakened by a `notify_one()` or `notify_all()`, it must re-acquire the mutex. C++ condition variables use **Mesa semantics**, meaning that between the time a thread is awakened and the time it actually re-acquires the lock, another thread might have sneaked in and changed the shared state.

Because of this, and to protect against "spurious wakeups" (the OS waking a thread for no reason), **you must always re-check the condition in a `while` loop or use the predicate version of `wait`**:

```
std::unique_lock<std::mutex> lock(mtx);
// The lambda predicate ensures the thread goes back to sleep if the buffer is still empty
cv.wait(lock, [] { return !buffer.empty(); });
```

1.3 Optimization: Notifying Outside the Lock

To minimize contention, it is a best practice to notify the condition variable *after* releasing the lock.

```
{
    std::lock_guard<std::mutex> lock(mtx);
    buffer.push_back(data);
} // Mutex is released here
cv.notify_one(); // Awakened thread won't immediately block trying to get the mutex
```

1.4 Graceful Termination (Sentinels)

To cleanly shut down consumers waiting on a condition variable, producers can insert a "Sentinel" value (e.g., `-1`) into the buffer. When a consumer pops the sentinel, it knows no more data will arrive and it can break its infinite loop and terminate.

---

2. Thread-Local Storage (`thread_local`)

When multiple threads need their own isolated instance of a variable (like a Random Number Generator, `std::mt19937`), sharing a global instance protected by a mutex creates a massive performance bottleneck.

The `thread_local` keyword ensures that each thread gets its own independent instance of the variable, allocated on the heap when the thread starts and destroyed when it terminates.

```
void simulate_work() {
    // Each thread gets its own RNG, seeded with its unique Thread ID
    thread_local std::mt19937 generator(std::this_thread::get_id());
    // ... use generator safely without locks
}
```

---

3. The Readers-Writers Problem

In many applications, a shared resource is read frequently but modified rarely. Using a standard `std::mutex` forces readers to wait for each other, destroying parallelism.

C++ provides `std::shared_mutex` to solve this:

- **Readers (`std::shared_lock<std::shared_mutex>`):** Multiple threads can hold a shared lock simultaneously.
- **Writers (`std::unique_lock<std::shared_mutex>`):** Requires exclusive access. It blocks if there are any active readers, and blocks new readers from entering.

Warning: The C++ standard does not guarantee fairness. A continuous stream of readers can starve a waiting writer indefinitely.

---

4. One-Shot Synchronization: Promises and Futures

Sometimes, threads only need to communicate a single value or signal a one-off event (like an alarm clock). Using mutexes and condition variables for this is overkill. C++ provides a single-assignment communication channel via `std::promise` and `std::future`.

- **`std::promise<T>`:** The "write" end of the channel. It is move-only.
- **`std::future<T>`:** The "read" end of the channel. Calling `.get()` blocks the thread until the promise is fulfilled. `.get()` can only be called once.

```cpp
#include <iostream>
#include <thread>
#include <future>

void worker(std::promise<int> prom) {
    // ... do heavy computation ...
    prom.set_value(42); // Fulfills the promise
}

int main() {
    std::promise<int> p;
    std::future<int> f = p.get_future();

    // Promise is move-only
    std::thread t(worker, std::move(p));

    // Blocks until the worker calls set_value
    std::cout << "Result: " << f.get() << "\n";
    t.join();
    return 0;
}
```

(Note: If multiple threads need to read the same future, you must use `std::shared_future`.)

---

5. Building a Generic Thread Pool

Spawning and destroying threads for every small task introduces unacceptable OS overhead. A **Thread Pool** solves this by spawning a fixed number of worker threads at startup. These workers sit in an infinite loop, pulling tasks from a shared, thread-safe queue.

5.1 Type Erasure and `std::packaged_task`

The challenge of a thread pool is that it must accept tasks with different signatures and return types. To put them in a single `std::queue`, we must erase their types.

We achieve this by wrapping functions in `std::packaged_task<ReturnType(Args...)>`, binding arguments using lambdas or `std::bind`, and casting them down to a generic `std::function<void()>`.

The `packaged_task` automatically ties the function's execution to a `std::future`, allowing the caller to retrieve the result later without the Thread Pool needing to know what the return type was.

---

6. Task-Based Parallelism with `std::async`

For spontaneous asynchronous tasks where a full Thread Pool isn't required, C++ provides `std::async`. It abstracts away thread creation and automatically returns a `std::future`.

6.1 Execution Policies

- `std::launch::async`: Forces the runtime to spawn a new thread immediately.
- `std::launch::deferred`: Lazy evaluation. The task is executed synchronously only when `.get()` is called on the future.

- If no policy is specified, the implementation chooses (often preferring deferred if system resources are low).

6.2 The Temporary Future Trap

A critical pitfall in C++ is that the destructor of a `std::future` returned by `std::async` **blocks** until the asynchronous operation completes.

```
// DANGEROUS: This runs sequentially!
for(int i = 0; i < 5; i++) {
    // The returned future is a temporary object. It is destroyed immediately
    // at the semicolon, blocking the loop until the task finishes.
    std::async(std::launch::async, heavy_task, i);
}

// CORRECT: Store the futures to keep them alive
std::vector<std::future<void>> futures;
for(int i = 0; i < 5; i++) {
    futures.push_back(std::async(std::launch::async, heavy_task, i));
}
// Now they run in parallel.
```

6.3 Divide and Conquer & Backpressure

When applying `std::async` or a Thread Pool to recursive Divide and Conquer algorithms (like Merge Sort), you must implement **backpressure**. If every recursive split spawns new tasks, you will quickly exhaust system memory or blow up the task queue. The algorithm must track its recursion depth or the queue size, falling back to sequential execution when a certain threshold is reached.

20. Workload Balancing & Task Scheduling

This chapter explores the critical challenge of workload balancing in data-parallel computations. Achieving peak parallel performance requires an optimal distribution of tasks among workers, forcing a constant trade-off between maximizing data locality and minimizing workload imbalance.

1. The Workload Imbalance Problem

Data Parallelism involves applying a function to multiple partitions of a dataset using a set of Workers. If the partitions do not require the exact same amount of computation time, the system suffers from **Workload Imbalance**.

- **The Bottleneck:** In a synchronous parallel execution, the completion time is strictly determined by the slowest worker. If one thread finishes its partition in 1 second while another takes 10 seconds, the first thread sits idle for 9 seconds, wasting CPU resources and destroying scalability.
- **The Trade-off:** To fix imbalance, we can divide the data into many smaller tasks and distribute them flexibly. However, doing so destroys **Data Locality** (leading to cache misses) and introduces **Scheduling Overhead** (mutex contention, atomic operations).

1.1 Skewed Workloads: The Mandelbrot Set

The computation of the Mandelbrot set is a classic example of a highly skewed workload. Each pixel is computed independently via an iterative mathematical formula. * "Black" pixels (inside the set) reach the maximum number of iterations, taking a long time to compute. * Colored pixels (outside the set) escape the formula quickly, requiring very few iterations. If we assign rows of the image to threads using naive static partitioning, the thread that gets the central rows (containing mostly black pixels) will take orders of magnitude longer to finish than the threads processing the top or bottom edges.

---

2. Static Distribution Policies

Static policies partition the dataset and assign tasks to threads before the computation begins. They introduce zero runtime overhead but cannot adapt if the workload proves to be irregular.

2.1 Block Distribution

The dataset is divided into contiguous chunks of size $B = \lceil N/P \rceil$, where N is the number of elements and P is the number of threads. * **Pros:** Maximizes spatial locality (great for caching). Minimal overhead. * **Cons:** Terrible for skewed workloads. * **Best for:** Regular computations like Vector Addition or Matrix-Vector multiplication.

```
// Static Block Partitioning Logic
int chunk_size = (N + num_threads - 1) / num_threads;
int start = thread_id * chunk_size;
int end = std::min(start + chunk_size, N);

for (int i = start; i < end; ++i) {
    // compute contiguous block
}
```

2.2 Cyclic Distribution

Elements are assigned to threads using a stride equal to the number of threads (e.g., Thread 0 gets elements 0, 4, 8; Thread 1 gets 1, 5, 9).

- **Pros:** Naturally balances skewed workloads by "shuffling" the heavy tasks across all threads.
- **Cons:** Completely destroys spatial locality, leading to massive cache misses.

2.3 Block-Cyclic Distribution

A hybrid approach. Instead of a stride of 1, we use a continuous chunk of size C, and then jump by a stride of $P \times C$.

This preserves a degree of locality within the block while still shuffling the workload globally.

---

3. Dynamic Task Assignment

For highly irregular or unpredictable workloads (e.g., Triangular Irregular Networks or computing the symmetric All-Pairs Distance Matrix for the MNIST dataset), static policies fail. We must use dynamic scheduling, where tasks are assigned to workers at runtime.

3.1 Work Sharing (Centralized Queue / Index)

All tasks are placed in a single, centralized shared structure (like a `std::queue` or a global atomic counter). Workers fetch a chunk of tasks, compute them, and then return to fetch more.

- **Implementation:** Instead of a complex queue, if tasks are just array indices, we can use a single `std::atomic<int>`.
- **Pros:** Achieves near-perfect load balancing. Fast workers automatically process more tasks than slow workers.
- **Cons:** The centralized structure becomes a bottleneck due to lock contention or atomic access scaling poorly with high thread counts.

```cpp
#include <atomic>
#include <vector>

std::atomic<int> global_index{0};
const int chunk_size = 16;

void dynamic_worker(int total_tasks) {
    while (true) {
        // Fetch and increment atomically
        int start = global_index.fetch_add(chunk_size);

        if (start >= total_tasks) break; // No more tasks

        int end = std::min(start + chunk_size, total_tasks);

        // Compute dynamically acquired chunk
        for (int i = start; i < end; ++i) {
            compute_heavy_task(i);
        }
    }
}
```

3.2 Work Stealing (Distributed Queues)

To solve the contention bottleneck of Work Sharing, **Work Stealing** gives each worker its own local task queue.

- A worker primarily fetches tasks from its own local queue (lock-free or low-contention).
- If a worker's local queue becomes empty, it becomes a "thief", randomly selecting another worker and stealing a chunk of tasks from the back of their queue.
- **Pros:** Highly scalable, minimal contention, preserves locality better than centralized queues.
- **Cons:** Complex to implement, especially regarding termination detection (how to know when all queues are empty and no tasks are currently flying across the network).

---

4. The Chunk Size Dilemma

When using Block-Cyclic or Dynamic approaches, choosing the right **Chunk Size** (Task Size) is critical and forms a U-shaped performance curve:

1. **Too Large:** Low scheduling overhead, but risks severe workload imbalance. If the last chunk happens to contain all the heaviest computations, one thread will be stuck while the others idle.
2. **Too Small:** Perfect load balancing, but the overhead dominates. If the chunk size is 1, threads spend more time fighting over the `std::atomic` index or the mutex than actually computing the data.

The Golden Rule: There is no universal formula to calculate the optimal chunk size. It must be found experimentally (trial and error) by tuning the parameter until the overhead and the load imbalance are perfectly balanced for the specific hardware and dataset.

21. Theoretical Models of Computation

This chapter abstracts away from the physical hardware (like cache hierarchies and NUMA nodes) to explore the fundamental theoretical limits of parallel execution. By utilizing idealized models, we can establish strict

upper and lower bounds on performance and mathematically prove the efficiency of parallel algorithms.

1. The Work-Span (Work-Depth) Model

The Work-Span model represents a parallel program as a Directed Acyclic Graph (DAG). * **Nodes** represent discrete tasks (with an associated execution cost). * **Edges** represent strict data dependencies between tasks.

A task becomes "ready" to fire only when all its predecessors have completed. The model assumes a **Greedy Scheduler**: if a task is ready and a processor is available, the task is dispatched immediately with zero overhead.

1.1 Fundamental Metrics: T_1 and T_∞

To evaluate the DAG, we define two critical metrics: * **Work (T_1):** The total amount of computation. It is the sum of the execution times of all tasks in the graph. This corresponds to the execution time on a single processor. * **Span or Critical Path (T_∞):** The longest weighted path of dependencies from the start node to the end node. It represents the absolute minimum execution time achievable, even with an infinite number of processors.

1.2 Lower Bounds on Execution Time

Given P available processors, the parallel execution time T_P is strictly bounded by two realities: 1. **The Work Bound:** $T_P \geq \frac{T_1}{P}$. The workload cannot be divided more perfectly than perfectly. 2. **The Span Bound:** $T_P \geq T_\infty$. You cannot compute a sequence of strictly dependent tasks any faster than sequential execution allows.

1.3 Brent's Theorem

Brent's Theorem provides a powerful upper bound on the execution time when using a finite number of processors P, assuming a greedy scheduler. It states:

$$T_P \leq \frac{T_1 - T_\infty}{P} + T_\infty$$

This formula mathematically isolates the parallelizable portion of the graph ($(T_1 - T_\infty)$) and divides it perfectly across P processors, while adding back the strict sequential bottleneck (T_∞).

Available Parallelism: The ratio $\frac{T_1}{T_\infty}$ dictates the potential concurrency. If we over-decompose a problem so that the Work is massively larger than the Span ($\frac{T_1}{T_\infty} \gg P$), the algorithm can achieve near-linear speedup. Modern runtimes like Intel TBB are built entirely around satisfying this model.

---

2. The PRAM (Parallel Random Access Machine) Model

While Work-Span ignores memory entirely, PRAM models an idealized shared-memory system.

- **Architecture:** P processors connected to a globally shared memory.
- **Uniform Access:** Every processor can access any memory location in exactly $O(1)$ constant time (no caches, no NUMA penalties, no bus contention).
- **Lockstep Execution:** All processors execute synchronously. In a single step, every processor performs:
  1. **Read** from shared memory into a local register.
  2. **Compute** an operation locally.
  3. **Write** the result back to shared memory.

2.1 Cost Optimality

In the PRAM model, minimizing absolute execution time is not enough. We must evaluate the **Cost** of the algorithm:

$$\text{Cost} = \text{Time} \times \text{Processors}$$

An algorithm is **Cost-Optimal** if its parallel Cost matches the time complexity of the best known sequential algorithm. For example, if a sequential algorithm runs in $O(N)$ time, a PRAM algorithm running in $O(\log N)$ time using N processors has a Cost of $O(N \log N)$. This is *not* cost-optimal, meaning processor efficiency is being wasted.

---

3. Parallel Prefix Sum (Scan)

To demonstrate PRAM algorithm design, we analyze the Prefix Sum, a foundational building block for many complex parallel algorithms (like quicksort or tree traversals).

Given an array X and an associative operator $\oplus$ (e.g., addition), the inclusive prefix sum calculates an array S where $S_i = X_0 \oplus X_1 \oplus \cdots \oplus X_i$.

3.1 Naive Recursive Doubling (Non-Cost-Optimal)

We assign N processors to an array of size N. At step k, each processor i reads its current value and the value at index $i - 2^{k-1}$, sums them, and writes the result.

- **Time:** $O(\log N)$ steps.
- **Processors:** N.
- **Cost:** $O(N \log N)$. Since the sequential prefix sum runs in $O(N)$, this parallel version is too fast but wastes resources; it is not cost-optimal.

3.2 The Cost-Optimal Algorithm

To achieve cost optimality, we restrict the number of processors to $P = \frac{N}{\log N}$.

1. **Local Sequential Scan:** Divide the array into P chunks of size $\log N$. Each processor computes the prefix sum of its local chunk sequentially. Time: $O(\log N)$.
2. **Parallel Prefix on Partial Sums:** Take the final value of each of the P chunks and perform the recursive doubling algorithm on just these P elements. Time: $O(\log P) \approx O(\log N)$.
3. **Local Updates:** Each processor adds the global accumulated value from Step 2 to all elements inside its local chunk. Time: $O(\log N)$.

- **Total Time:** $O(\log N) + O(\log N) + O(\log N) = O(\log N)$.
- **Cost:** $O(\log N) \times \frac{N}{\log N} = O(N)$. The algorithm is now perfectly cost-optimal.

4. Practical Application: Sparse Array Compaction

Theoretical PRAM algorithms dictate the implementation of actual standard libraries. The C++ `<numeric>` library provides `std::inclusive_scan` and `std::exclusive_scan` (which support `std::execution::par` execution policies).

A classic use case is **Sparse Array Compaction**. If you have a massive array mostly filled with zeros, you want to extract only the non-zero elements into a dense array. 1. Create a bitmap (1 if non-zero, 0 if zero). 2. Run a Parallel Prefix Sum on the bitmap. The resulting values perfectly dictate the target memory index in the new dense array for every non-zero element. 3. Scatter the non-zero values to their newly computed indices.